

# **Computer Hacking Forensic Investigator (CHFI)**

**Version 11**

## **Lab Report**

**Student Name:** Mayur Dipak Patil

**Course Name:** Computer Hacking Forensic Investigator (CHFI V11)

**Report Title:** CHFI Lab Report (Module 10)

**Instructor's Name:** Satish Singh

## **Table of Contents**

**Module 10: Investigating Web Attacks**

Lab 1: Identify and Investigate Web Application Attacks Using Splunk

Lab 2: Identify and Investigate Web Application Attacks Using Python

## **Module 10: Investigating Web Attacks**

### **Lab Scenario**

Forensic investigators need to investigate web attacks on a web server to identify the type of web attack and its severity. As part of this investigation, investigators must analyze the web server's log files, IDS, and SIEM tools which contain the IP address, browser, and system information of the attacker. An example of a severe, large-scale web attack would be a scenario where the website of a renowned banking organization is brought down by attackers and the entire data on it, including account holders' details, employers' and

employees' details, etc., is leaked online on the dark web. To investigate this web attack, investigators would need to analyze the IDS logs, web server logs, and SIEM alerts. By analyzing these details, they can determine the types of web attacks that have been attempted by the attackers. These may include cross-site scripting, SQL injection, bruteforce, and various other attacks. As an expert forensic investigator, you should know how to investigate web-related attacks.

## Lab Objectives

The objective of this lab is to help the students understand how to investigate various types of web attacks, including:

- Detection and analysis of an XSS attack by examining Apache logs, ModSecurity logs, and IIS logs
- Detection and analysis of a SQL injection attack by examining Apache logs
- Detection and analysis of a directory traversal attack by examining Apache logs, ModSecurity logs, and IIS logs
- Detection and analysis of a command injection attack by examining Apache logs
- Detection and analysis of an XXE attack by examining Apache logs
- Detection and analysis of a brute-force attack by examining Apache logs and IIS logs

## Overview of Investigating Web Attacks

Web attacks are of different types. A denial-of-service (DoS) attack can be cited as an example of a web attack. In DoS attacks, visitors to a website are denied access to any information or services available on the website. In such cases, visitors may report the unavailability of online services which they were trying to access. Attackers can also use other techniques, such as SQL injection, command injection, brute-force attacks etc., to exploit any vulnerable website. An indication of a web attack can be the redirection of a user to an unknown website. Unusually slow network performance and frequent server reboots may also indicate a web attack. In case a web attack is suspected, investigators need to examine the log data from various sources to understand where the attack originated and mitigate it at the earliest.

## Lab Tasks

Recommended lab to assist you in investigating web attacks:

- Identify and Investigate Web Application Attacks Using Splunk
- Identify and Investigate Web Application Attacks Using Python

## **Lab 1: Identify and Investigate Web Application Attacks Using Splunk**

### **Lab Scenario**

An e-commerce company has suffered a cyber-attack on its web application that lists and sells various goods/products. The attackers not only took down the website but also stole personally identifiable information of its users/customers. The company sought the services of the cyber forensics wing of the state law enforcement department to crack the

case, retrieve customer specific stolen information, secure its systems and all customer-related information, and thus help restore the customers' faith in their business.

To investigate the case and determine the types of web attacks that have been carried out by the attackers on the company's web application, Robert, the forensic investigator, now needs to examine and analyze log files from the Apache server, IIS server, and ModSecurity web application firewall using SIEM tool.

## Lab Objectives

Log files record events and activities that take place in an operating system or during the runtime of a computer software or service. Forensic examination of these log files helps the investigator identify malicious activities that take place on the web server.

The objective of this lab is to help you understand how to examine log files and look for artifacts pertaining to web application attacks with the SIEM tool.

## Overview of the Lab

This lab familiarizes you with the process of examining log files generated by servers and web application firewalls to check for indicators of web attacks using SIEM tools like Splunk Enterprise.

## Lab Tasks

1. Search Splunk in search bar as shown below:



2.Splunk Enterprise will now be launched through Brave. Upon the launch of the application through the browser, you will see 127.0.0.1:8000 in its address bar.

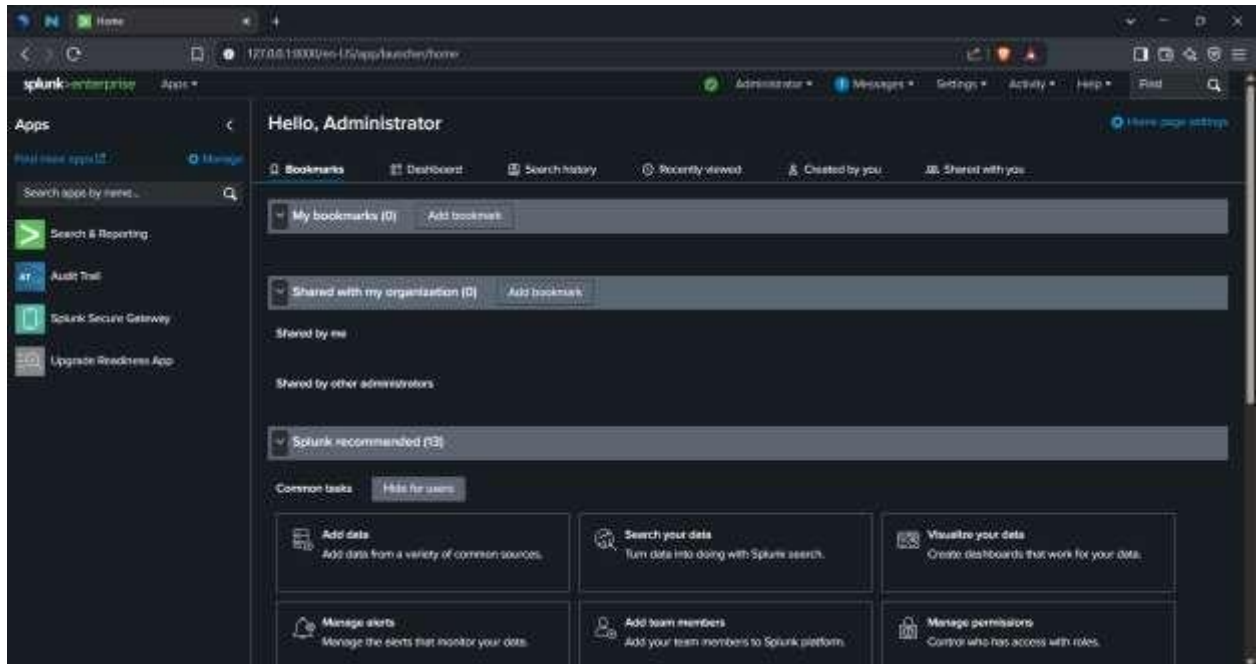
3.In the resulting web page, enter the Sign In credentials that you created for the Splunk Enterprise account during setup, and then click Sign In.



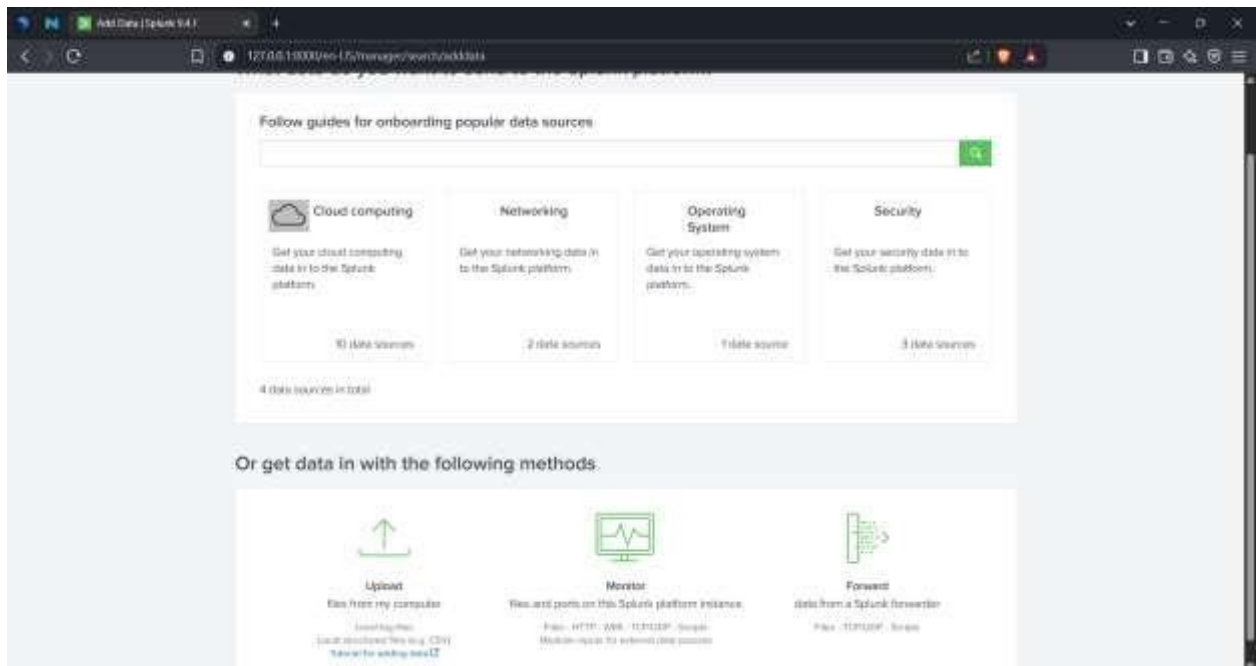
4. The above operation will sign you in to your Splunk Enterprise account.

5.The steps that follow from here on demonstrate the execution of our lab tasks after logging into the Splunk Enterprise account.

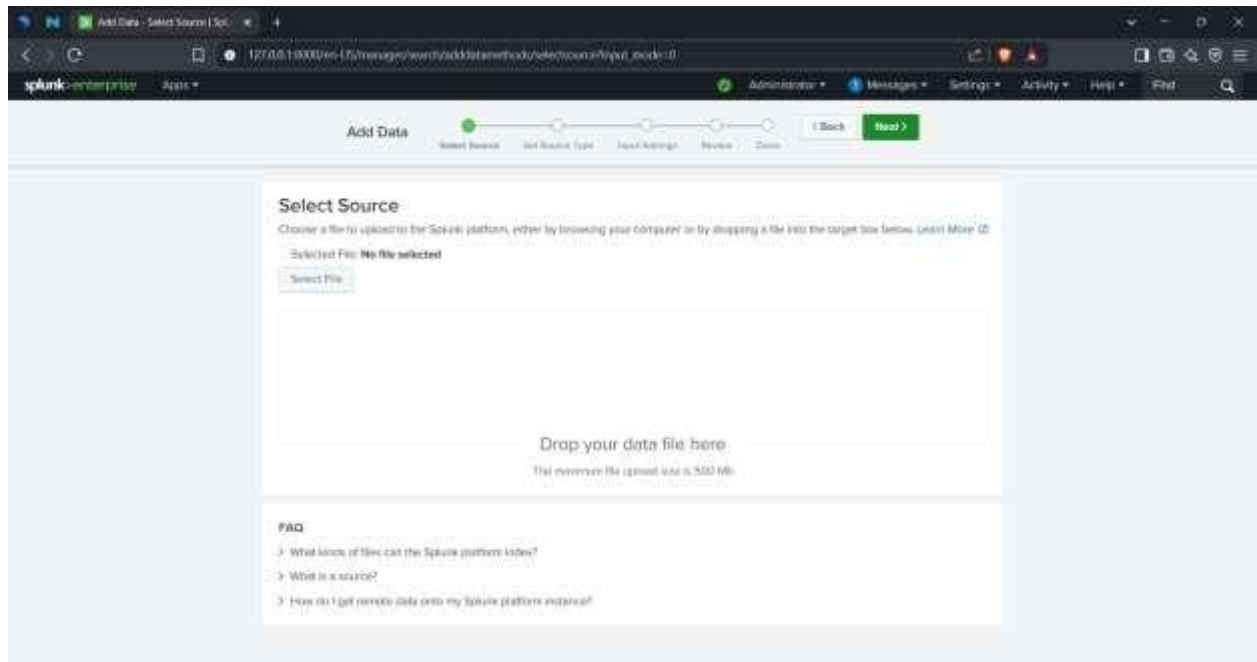
6.You will now see the homepage of Splunk Enterprise, where you will find the Add data option, as shown in the screenshot. Click Add Data.



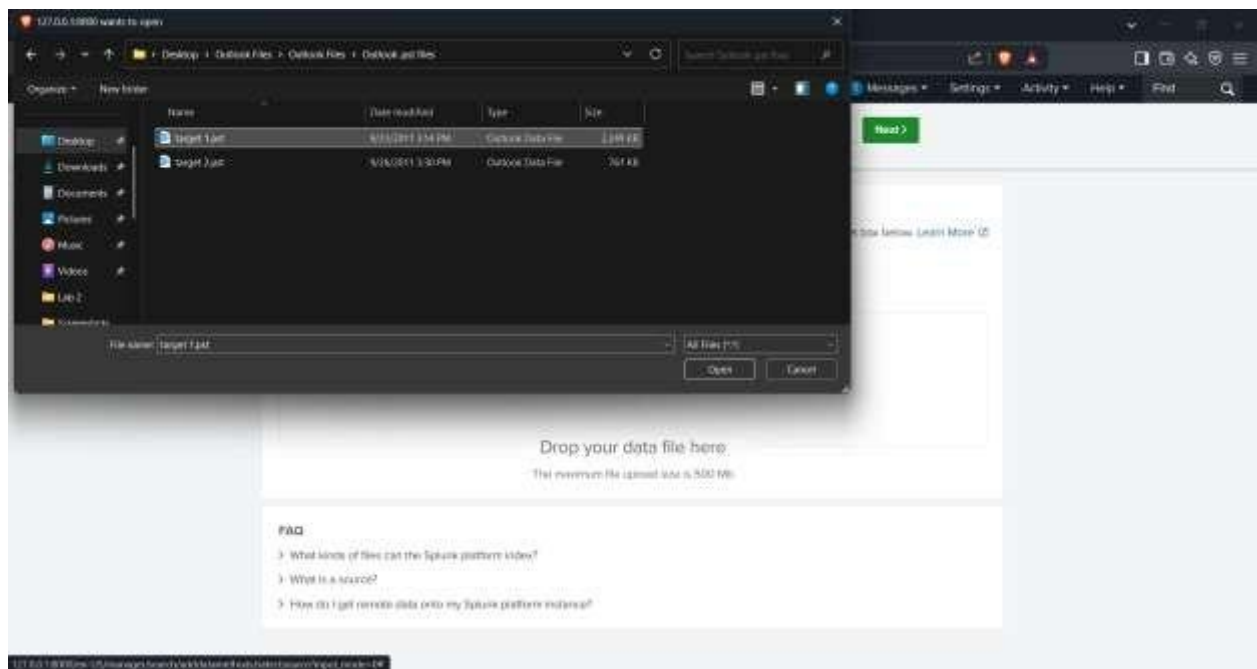
7. Now, scroll down the homepage to find and click the Upload files from my computer option.



8. The Select Source section will now appear. Click Select File, as shown in the screenshot below:

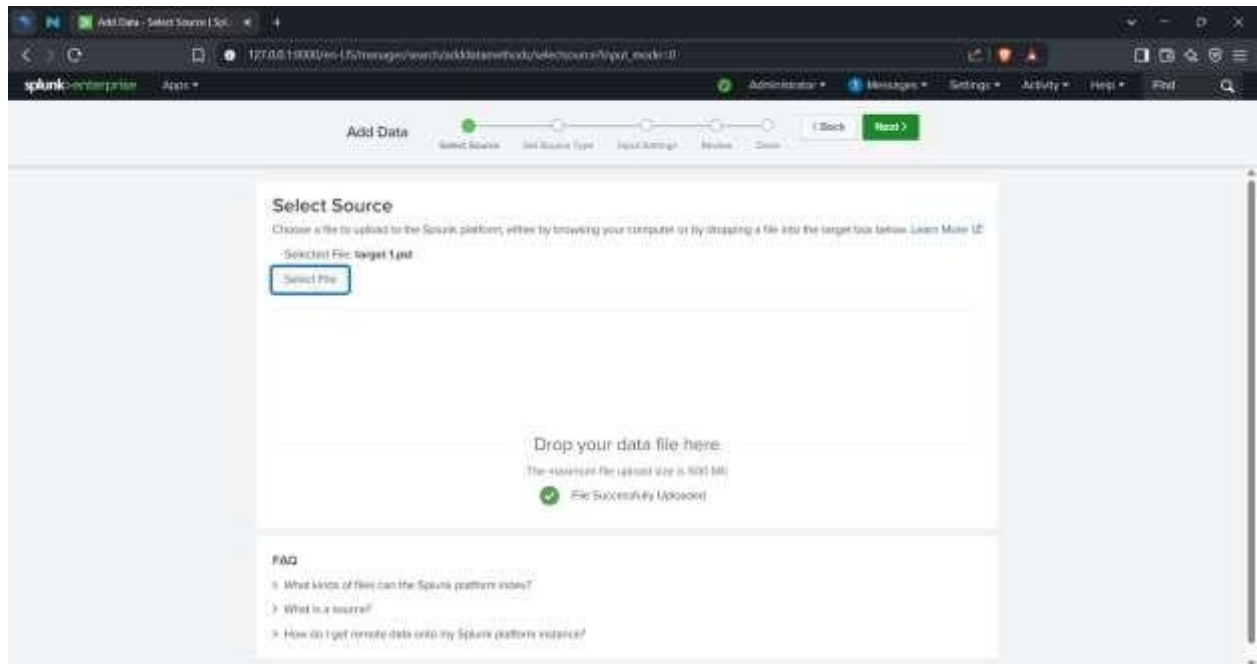


9. When an Open window appears, navigate to E:\CHFI-Tools\Evidence Files\Log Files\Apache Logs and select the Apache logs. Click Open to upload the file.

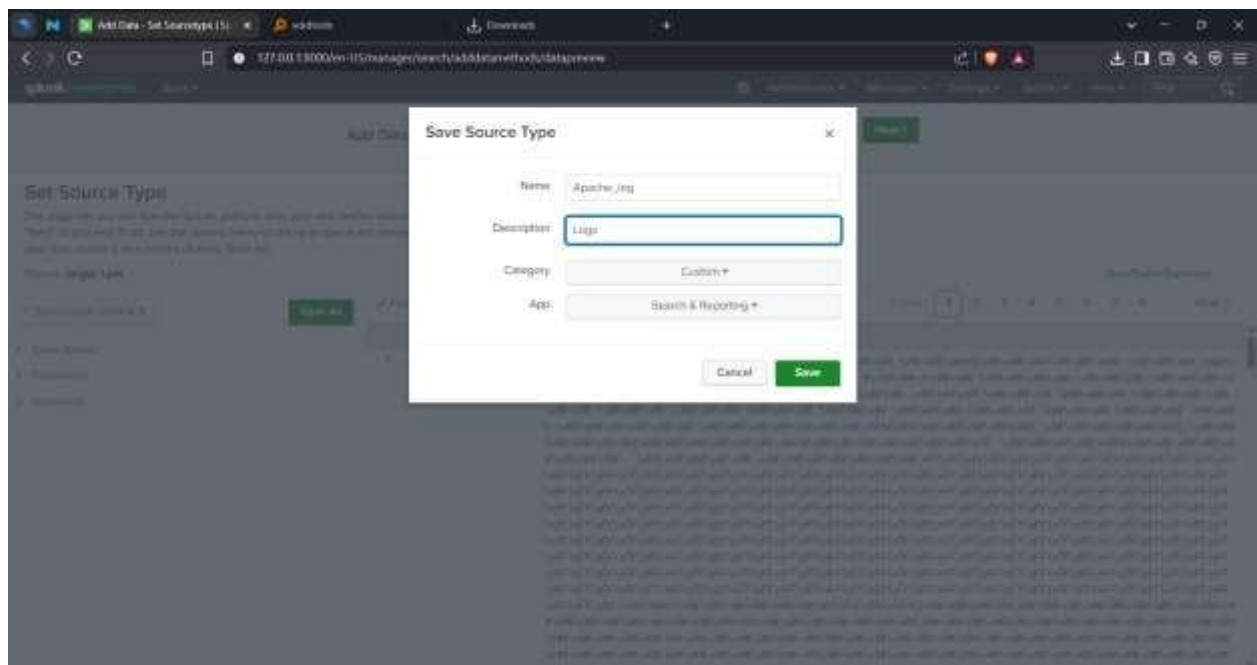


10. The Apache file will now be uploaded successfully. Click Next to continue.

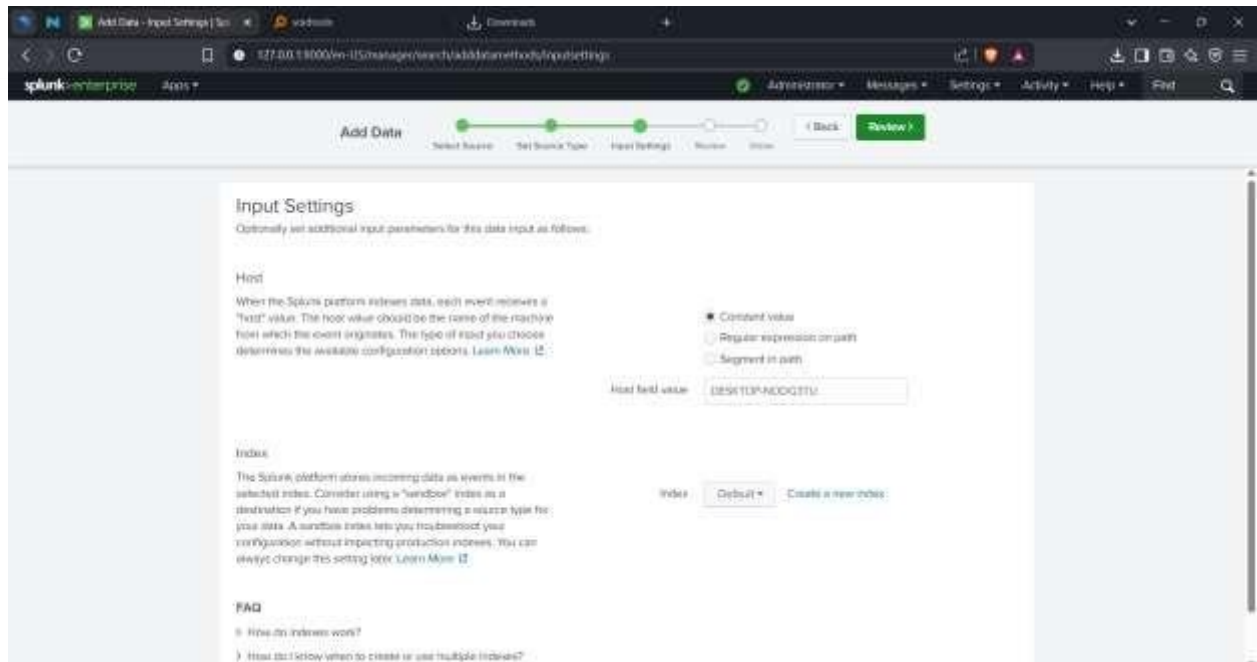




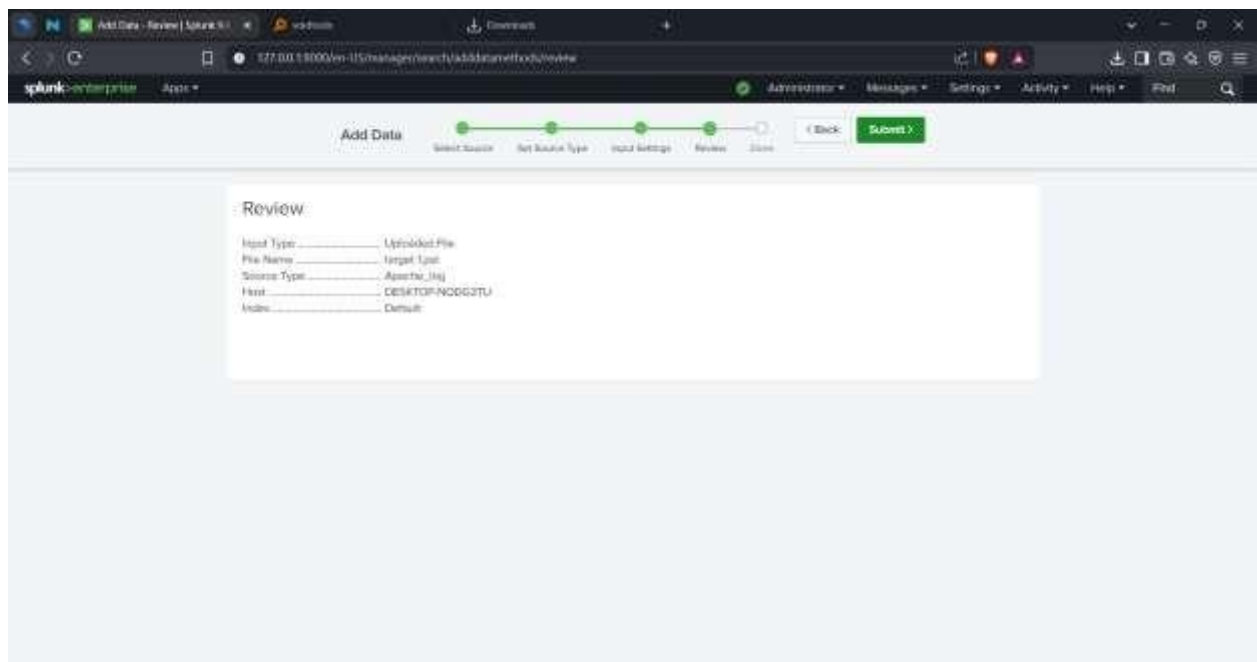
11. In the next step, you will see the Set Source Type section. Click Save As, as shown in the following screenshot:



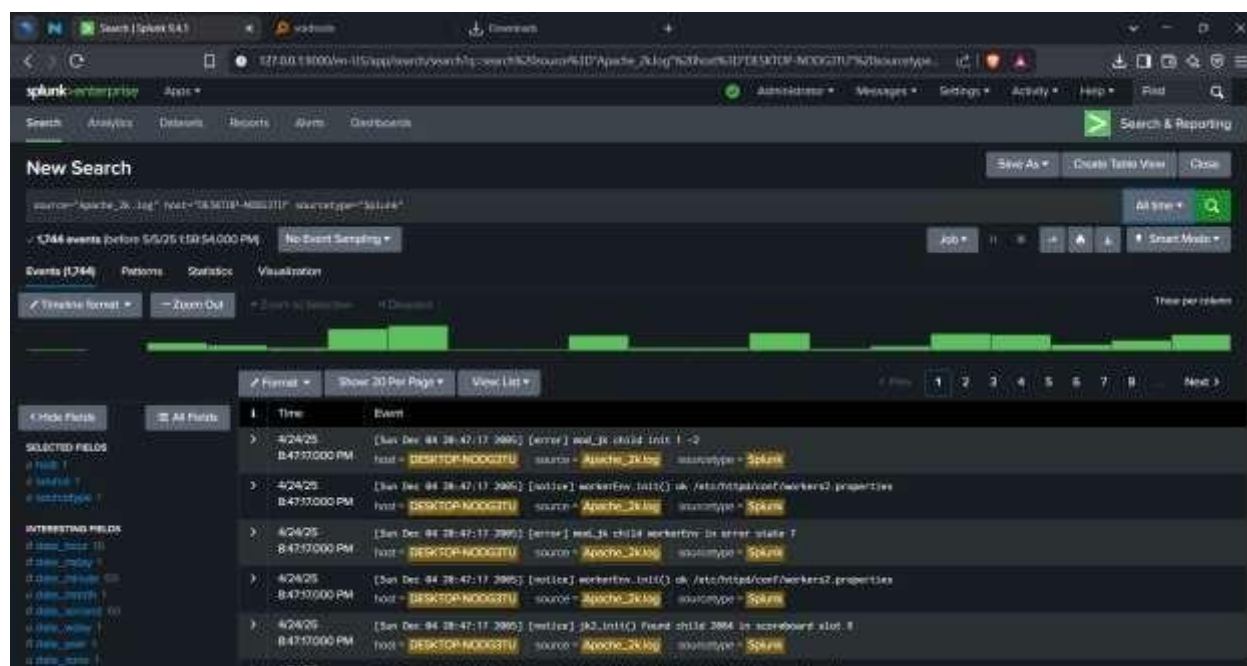
12. In the next step, you will see the Input Settings section. Click Review to proceed further.



13. In the next step, you will see the Review section. Ensure that the details under the Review section are correct and then click Submit to proceed further. If necessary, apply corrections to the details by revisiting the previous sections by clicking on Back, and then, come back again to the Review section and click Submit to proceed further.



14. You will now see the log entries from the uploaded file displayed in the Splunk Enterprise window under the Events tab, as shown in the screenshot below. Our task now is to examine the log file for signs of an Apache log.



15. In this manner, you can examine the log files from Apache as ModSecurity web application firewall for indicators pertaining to various web attacks.

## Lab 2: Identify and Investigate Web Application Attacks Using Python

### Lab Scenario

A cybercriminal assault has targeted an organization, compromising its web application specializing in selling diverse products. The malevolent actors not only disrupted the website's functionality but also illicitly acquired personally identifiable information from users. In response, the company engaged the cyber-forensics unit of the state law enforcement department to investigate, recover the pilfered customer data, fortify its systems, and restore customer confidence in the integrity of their business.

John is a forensic investigator, who is tasked with leveraging Python scripts to delve into server logs, detect attack patterns, and gather evidence to understand the nature of the attack.

### Lab Objectives

Investigating web application attacks involves comprehensive examination and analysis of cyber threats targeting web applications. This process employs various forensic techniques to detect malicious activities, uncover attack patterns, and collect evidence.

Digital investigators use tools and methodologies to scrutinize logs, trace vulnerabilities, and reconstruct attack scenarios.

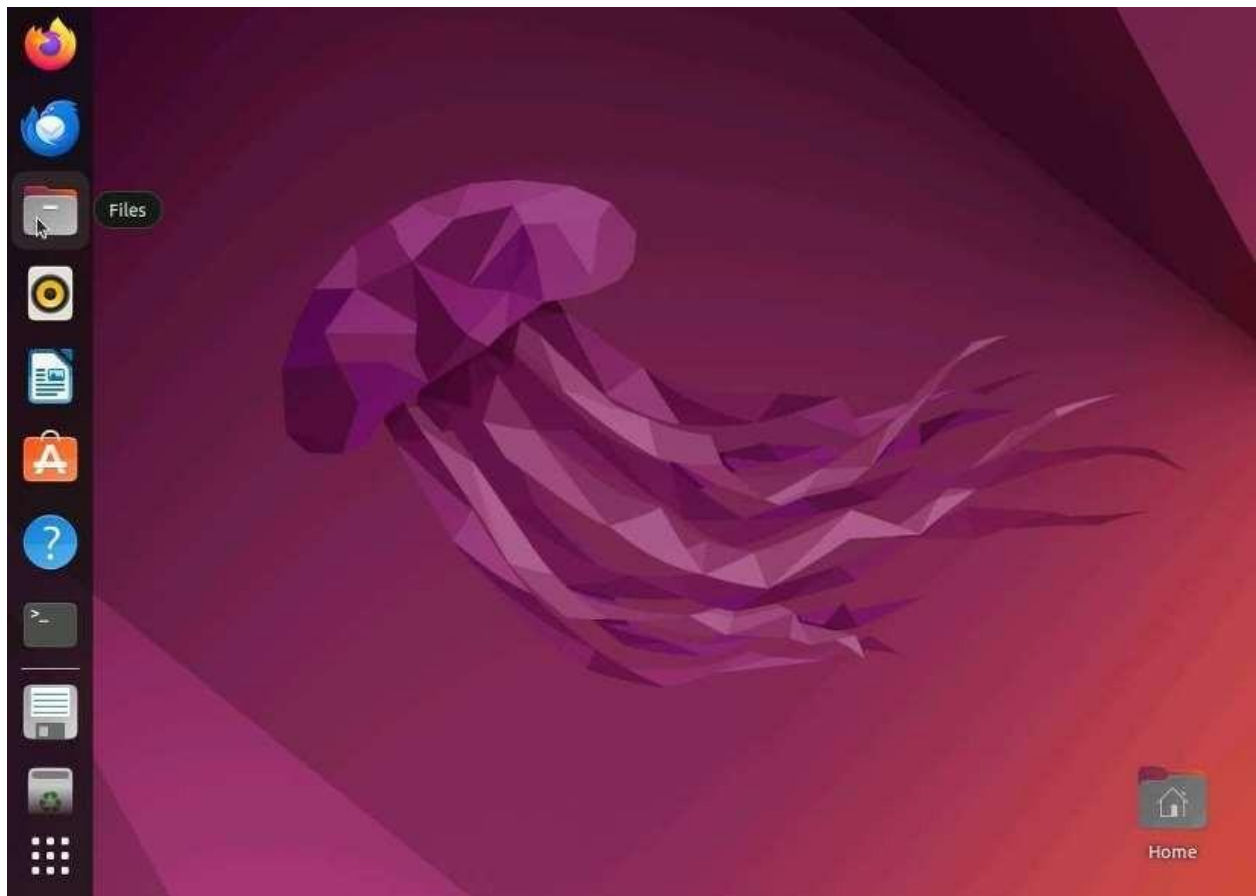
The objective of this lab is to help you understand how to examine log files and look for artifacts pertaining to web application attacks using Python script.

## Overview of the Lab

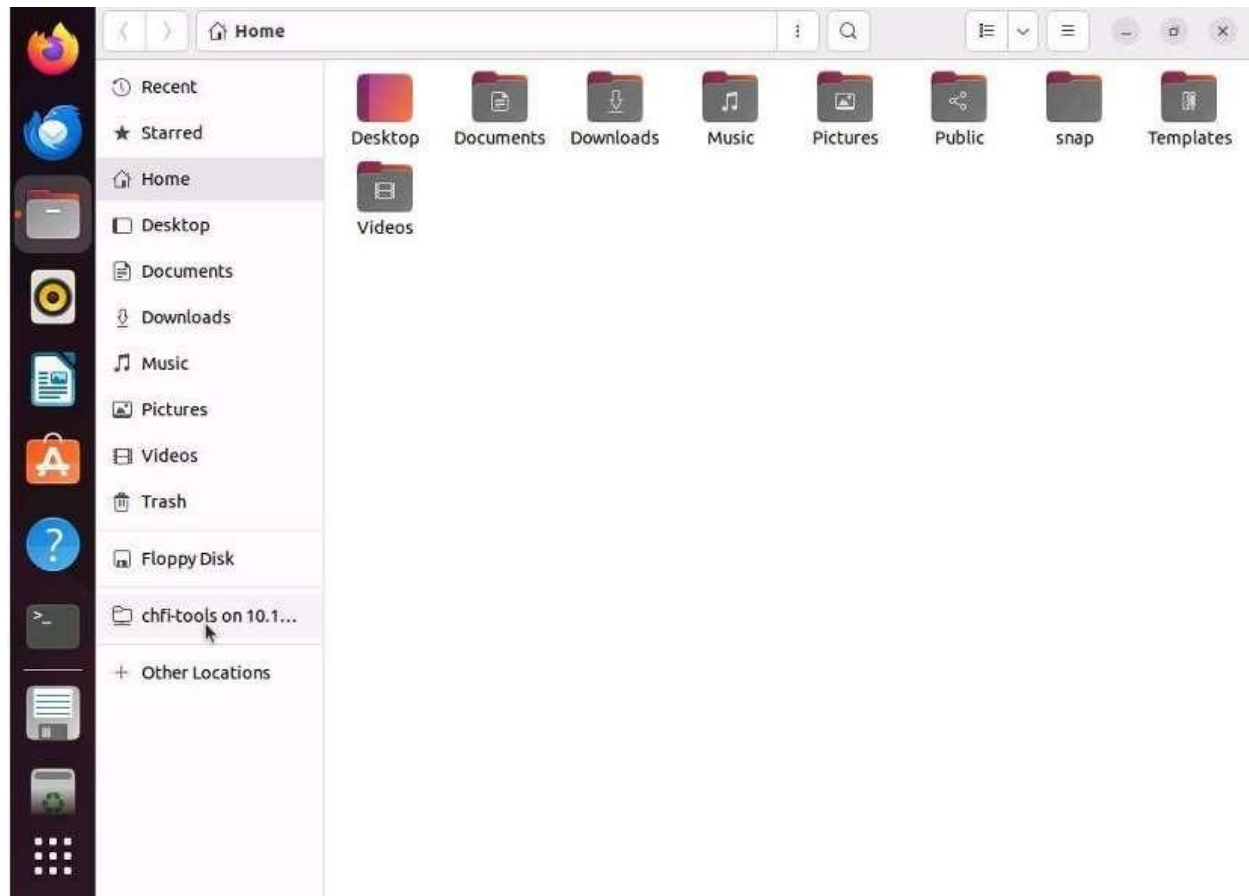
This lab familiarizes you with the process of examining log files generated by a web server, SQL injection logs, command injection and directory traversal logs to check for indicators of web attacks using Python scripts.

## Lab Tasks

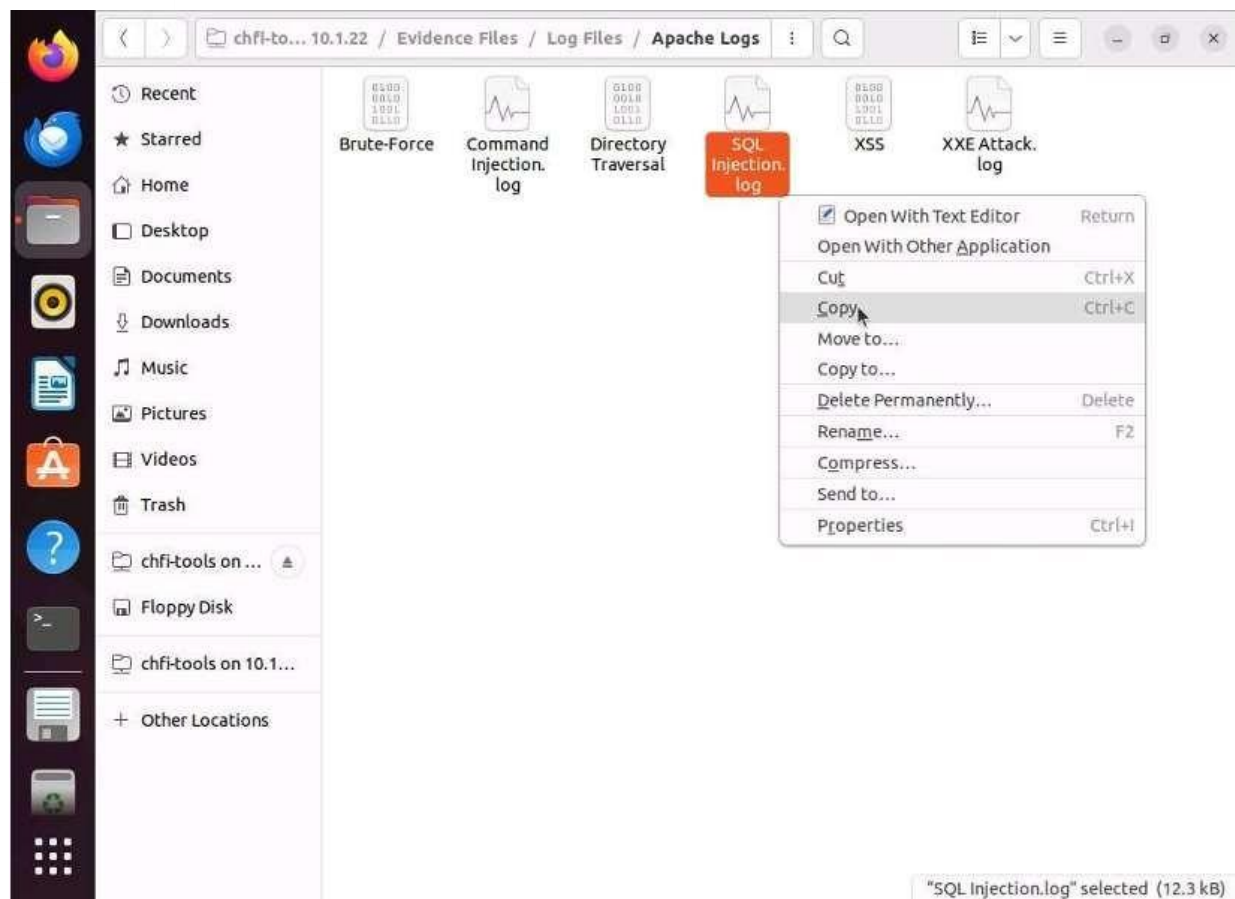
1. We will perform static analysis on the SQL Injection log file. To do so, we need to copy SQL Injection.log from the CHFI-Tools\Evidence Files\Log Files\Apache Logs directory and paste it in the Home directory.
2. Click on the Files icon in the Launcher panel to launch the File Manager.

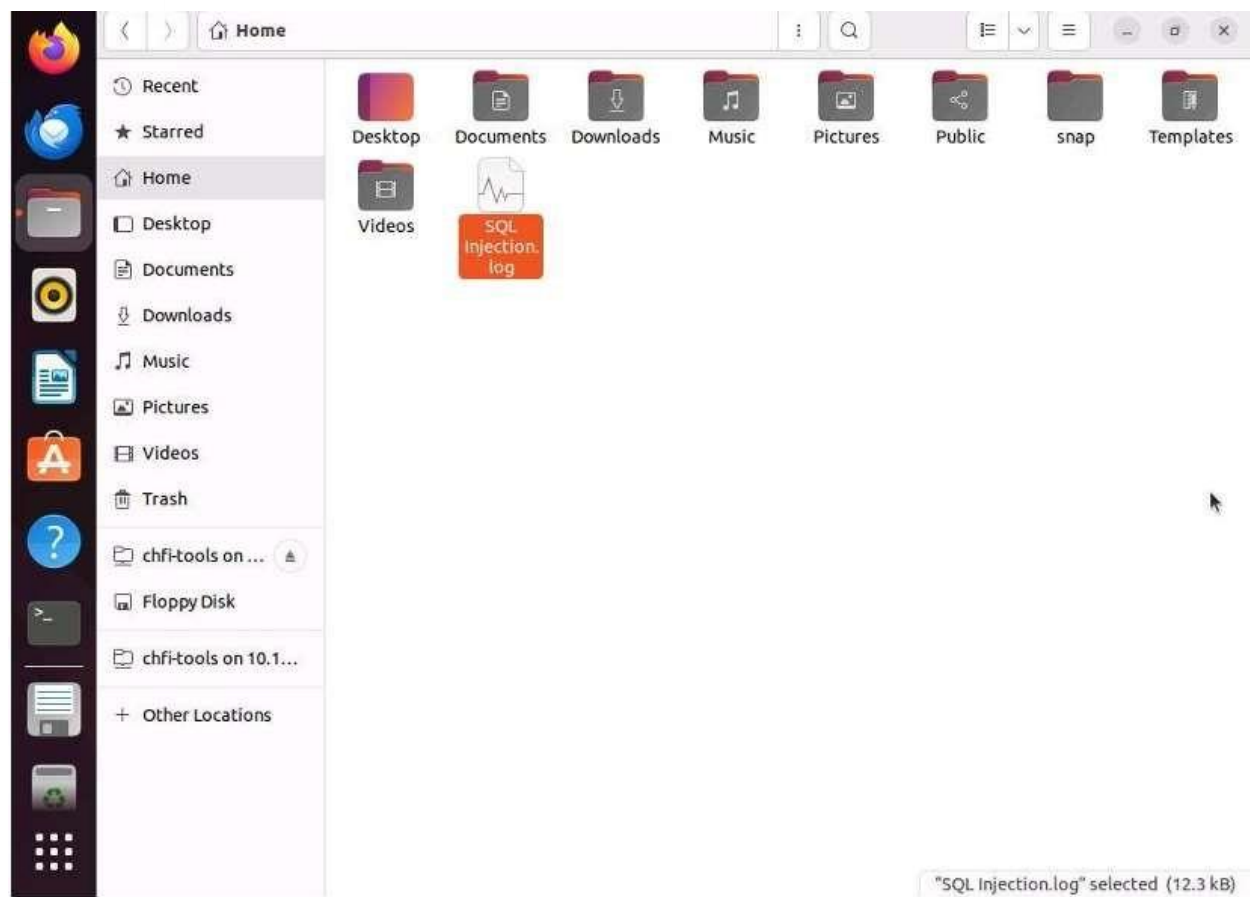


3.The File manager window will appear pointing to the Home directory. Click on the bookmarked chfi-tools on 10.10.1.22 directory

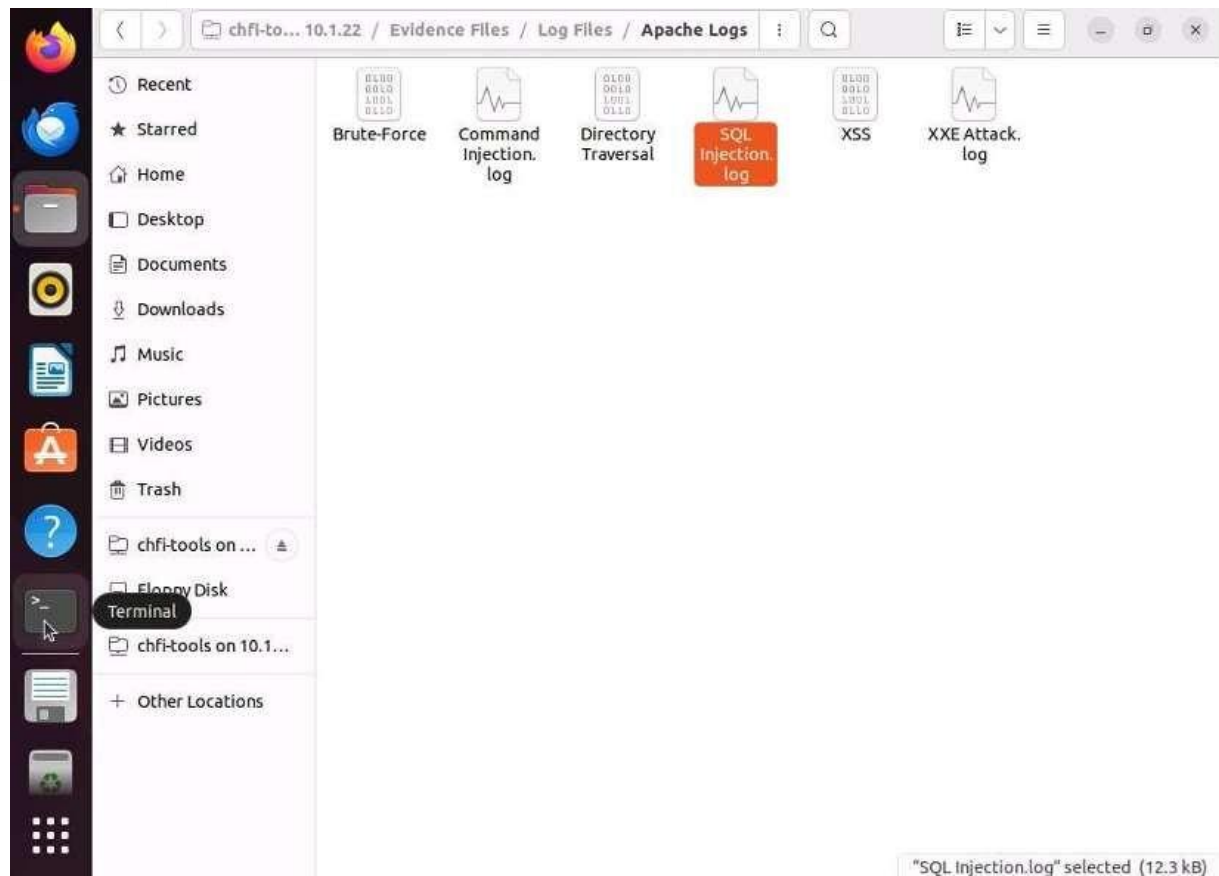


4.CHFI-Tools will appear in the window. Navigate to Evidence Files\Log Files\Apache Logs directory and copy the SQL Injection.log file and paste it in the Home directory





5. Now, we will write a Python script to analyze Apache web server logs. To do so, click on the Terminal icon from the launcher panel to launch the command line terminal.

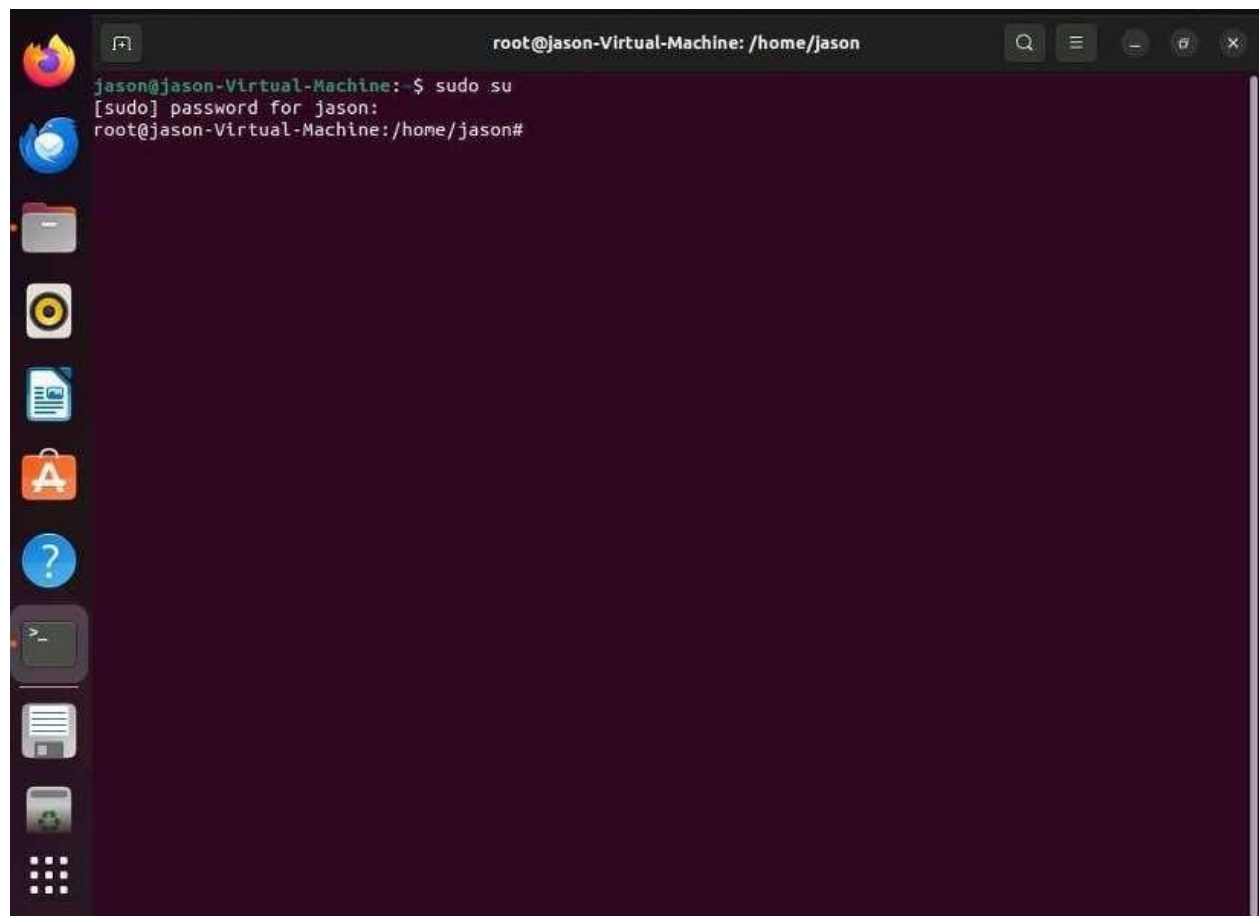


6.The command line terminal launches. You require root privileges in the terminal to install an application.

7. Now, type `sudo su` and press Enter. You will be prompted to enter the password. Type `toor` as the password and press Enter.

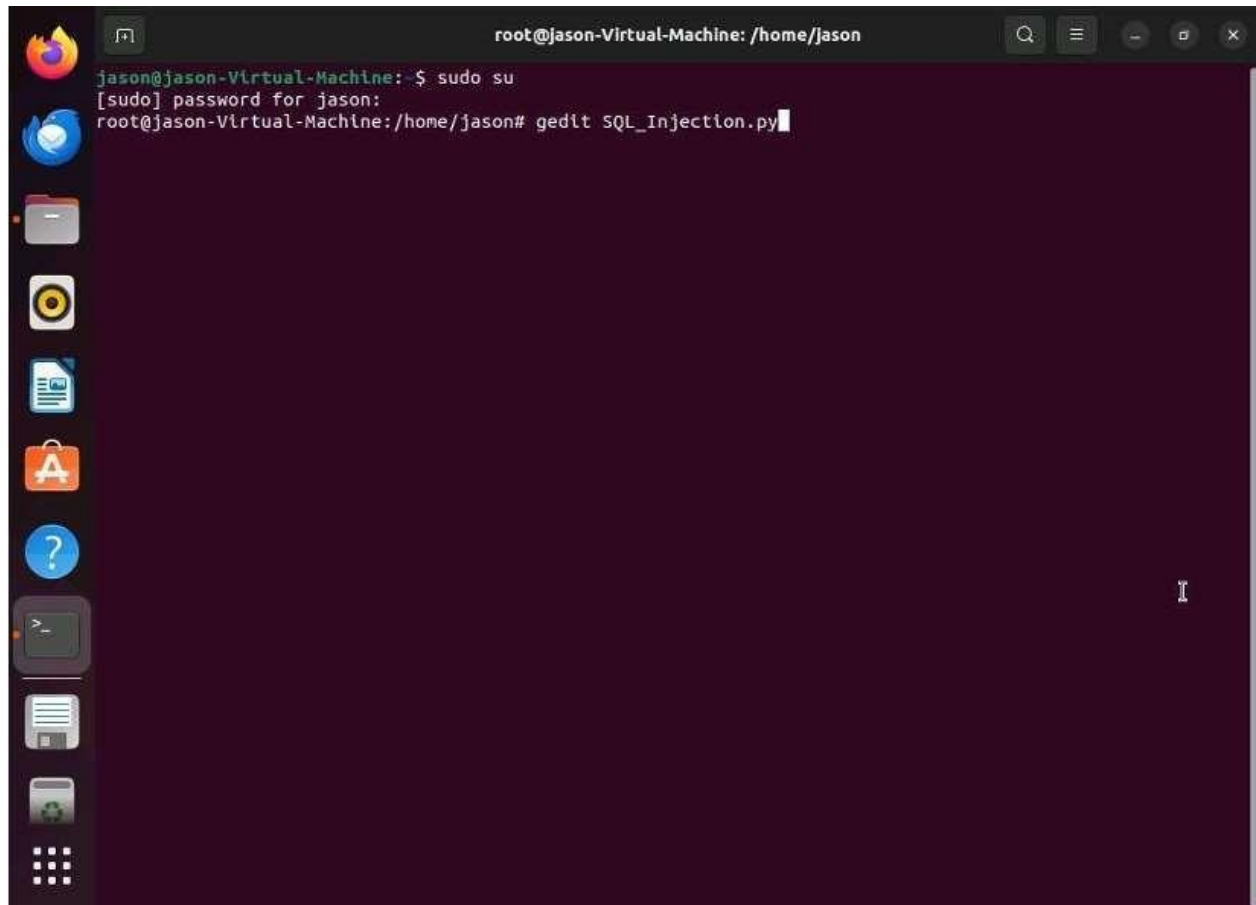
8.Now, you will enter the root terminal, as shown in the following screenshot:





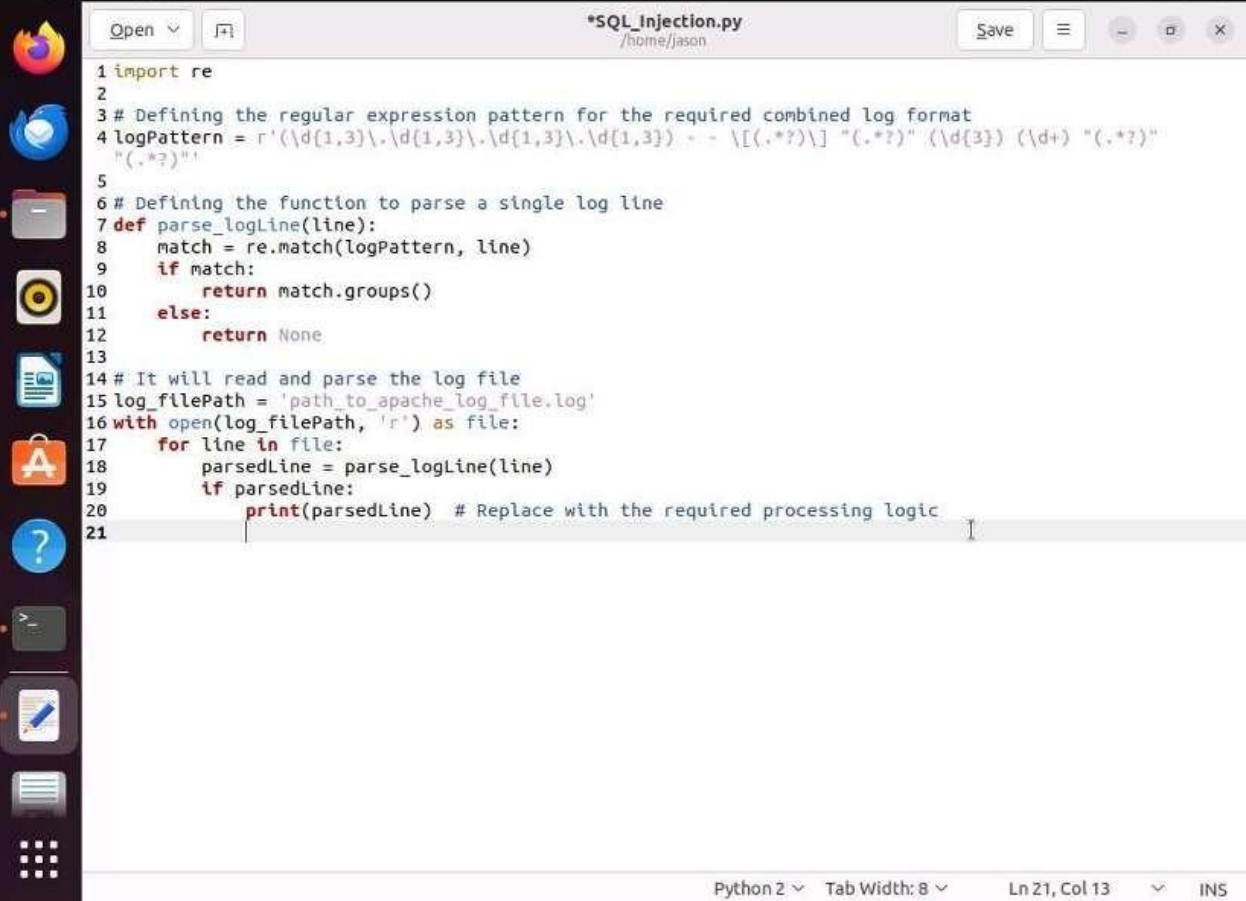
```
root@jason-Virtual-Machine: /home/jason
jason@jason-Virtual-Machine: $ sudo su
[sudo] password for jason:
root@jason-Virtual-Machine:/home/jason#
```

9. In the terminal window, type `gedit SQL_Injection.py` and press Enter to open a Text Editor window.



```
root@jason-Virtual-Machine: /home/jason
jason@jason-Virtual-Machine:~$ sudo su
[sudo] password for jason:
root@jason-Virtual-Machine:/home/jason# gedit SQL_Injection.py
```

10. The SQL\_Injection.py text editor will be opened. Now, type the code shown in the screenshot below into the file and save it in the Home directory.



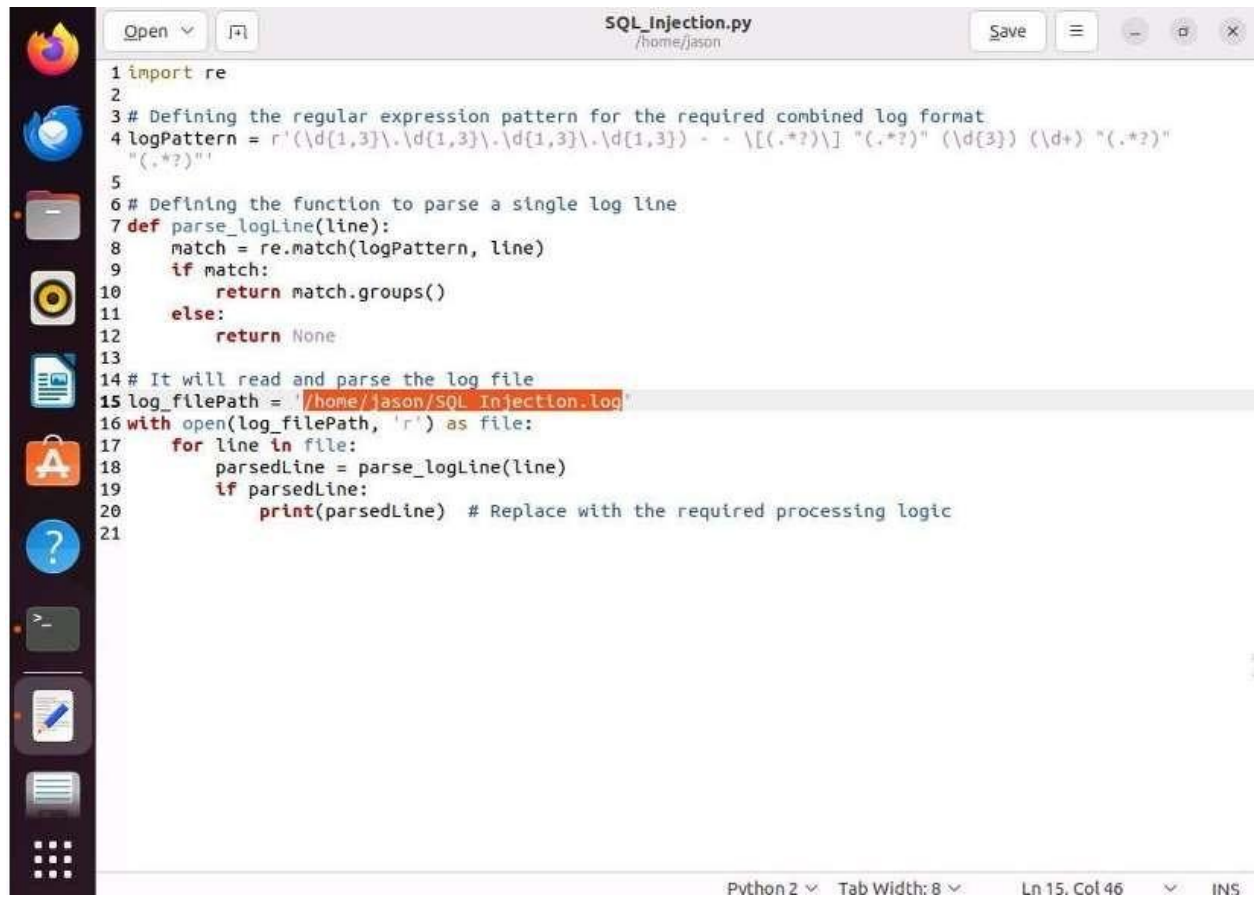
The screenshot shows a text editor window titled `*SQL_Injection.py` with the file path `/home/jason`. The editor contains a Python script with the following code:

```
1 import re
2
3 # Defining the regular expression pattern for the required combined log format
4 logPattern = r'(\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}) - - \[(.*?)\] "(.*?)" (\d{3}) (\d+) "(.*?)"'
5
6 # Defining the function to parse a single log line
7 def parse_logLine(line):
8     match = re.match(logPattern, line)
9     if match:
10         return match.groups()
11     else:
12         return None
13
14 # It will read and parse the log file
15 log_filePath = 'path_to_apache_log_file.log'
16 with open(log_filePath, 'r') as file:
17     for line in file:
18         parsedLine = parse_logLine(line)
19         if parsedLine:
20             print(parsedLine) # Replace with the required processing logic
21
```

The status bar at the bottom indicates the editor is using Python 2, with a tab width of 8, and the cursor is at line 21, column 13.

11. We need to provide the log file location in line 15 of the Python script to parse the logs.

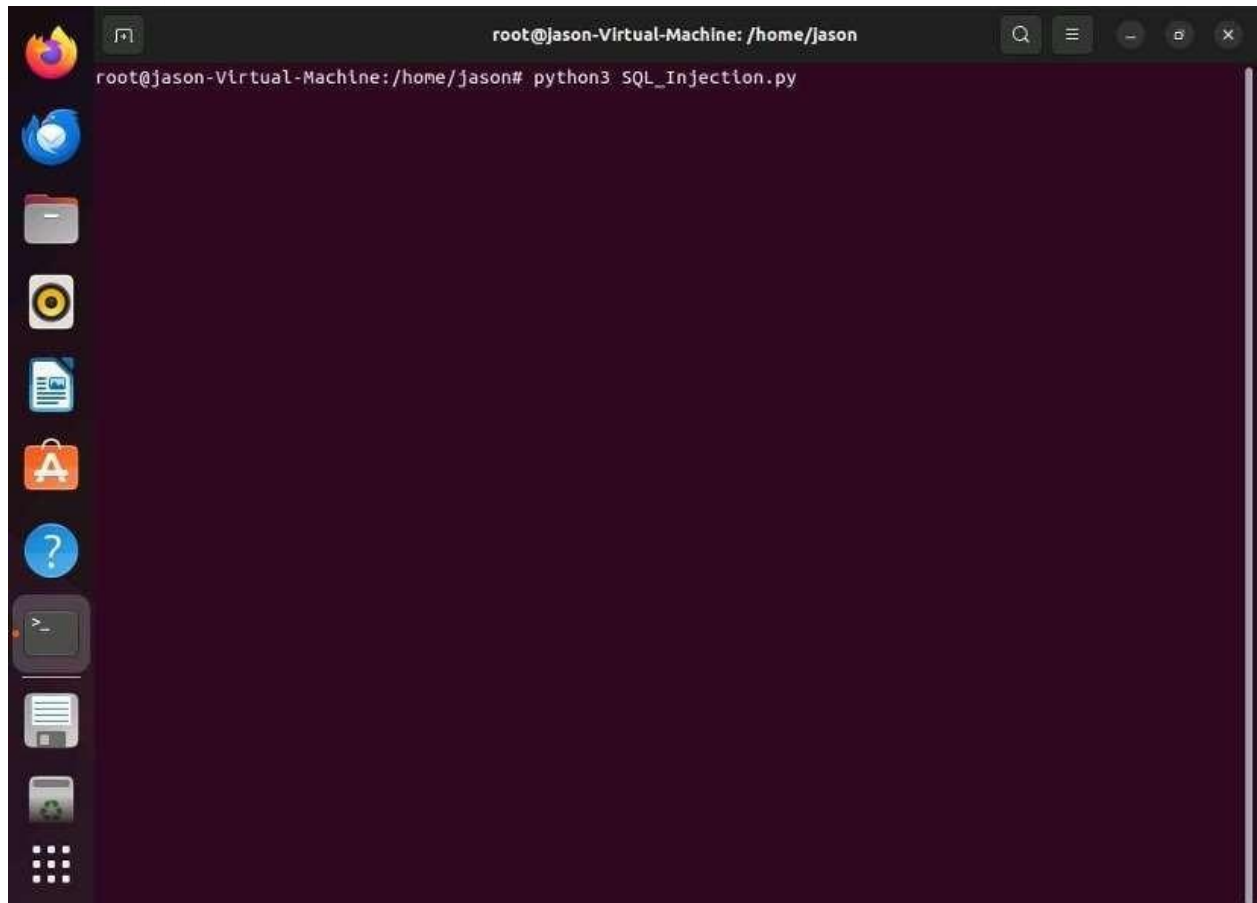
12. In line 15 replace `path_to_apache_log_file.log` with `/home/jason/SQL Injection.log` as shown in the screenshot. Save it and close the Text Editor window.



```
1 import re
2
3 # Defining the regular expression pattern for the required combined log format
4 logPattern = r'(\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}) - \[(.*?)\] "(.*?)" (\d{3}) (\d+) "(.*?)"'
5
6 # Defining the function to parse a single log line
7 def parse_logLine(line):
8     match = re.match(logPattern, line)
9     if match:
10         return match.groups()
11     else:
12         return None
13
14 # It will read and parse the log file
15 log_filePath = '/home/jason/SQL_Injection.log'
16 with open(log_filePath, 'r') as file:
17     for line in file:
18         parsedLine = parse_logLine(line)
19         if parsedLine:
20             print(parsedLine) # Replace with the required processing logic
21
```

Python 2 Tab Width: 8 Ln 15, Col 46 INS

13. Now, switch back to the terminal window, and type `python3 SQL_Injection.py` and press Enter to run the Python script.



14. Upon successful execution of the script, you can view the parsed SQL Injection logs in the terminal.

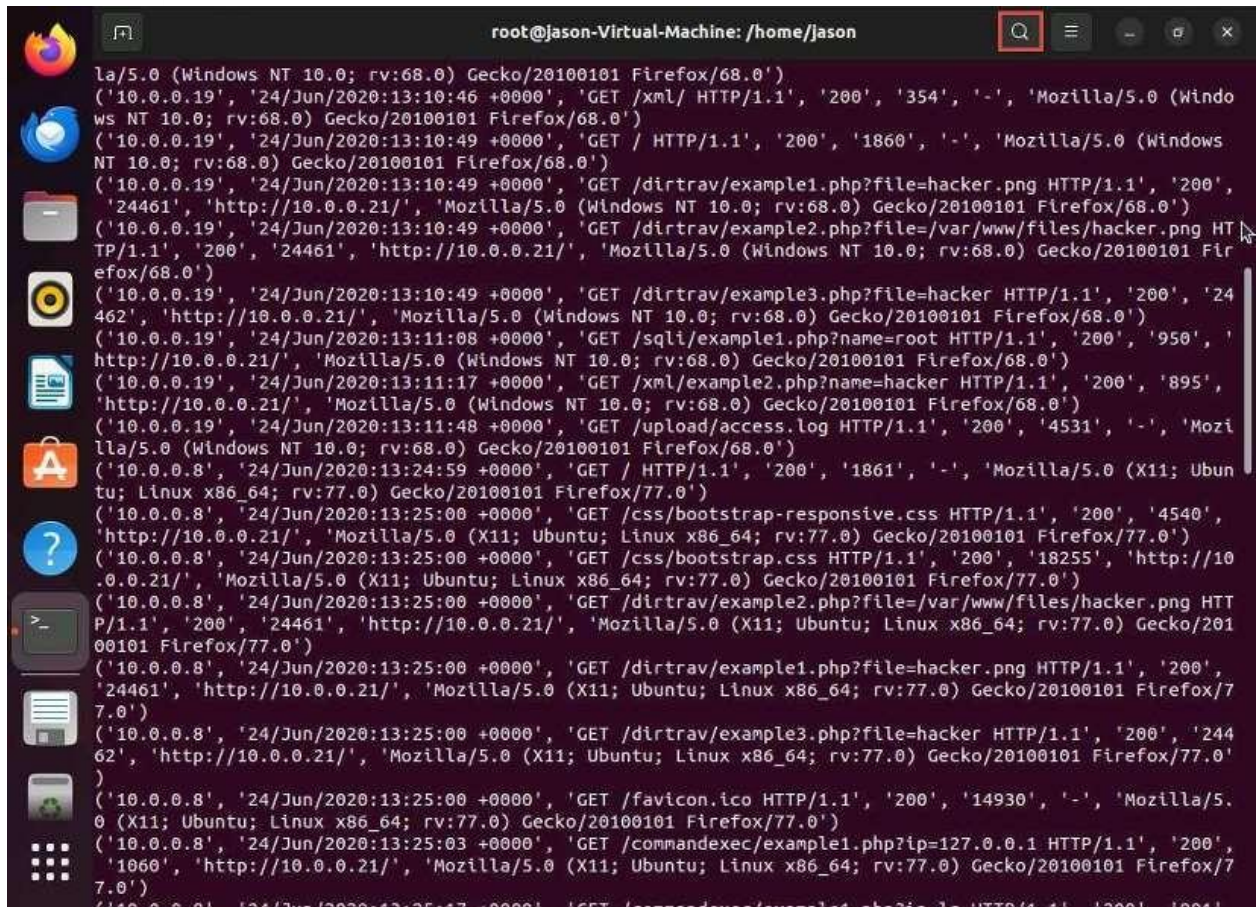
```

root@jason-Virtual-Machine: /home/jason
root@jason-Virtual-Machine:/home/jason# python3 SQL Injection.py
('10.0.0.19', '24/Jun/2020:12:42:29 +0000', 'GET / HTTP/1.1', '200', '1861', '-', 'Mozilla/5.0 (Windows NT 10.0; rv:68.0) Gecko/20100101 Firefox/68.0')
('10.0.0.19', '24/Jun/2020:12:42:29 +0000', 'GET /css/bootstrap-responsive.css HTTP/1.1', '200', '4540', 'http://10.0.0.21/', 'Mozilla/5.0 (Windows NT 10.0; rv:68.0) Gecko/20100101 Firefox/68.0')
('10.0.0.19', '24/Jun/2020:12:42:29 +0000', 'GET /css/bootstrap.css HTTP/1.1', '200', '18255', 'http://10.0.0.21/', 'Mozilla/5.0 (Windows NT 10.0; rv:68.0) Gecko/20100101 Firefox/68.0')
('10.0.0.19', '24/Jun/2020:12:42:29 +0000', 'GET /dirtrav/example2.php?file=/var/www/files/hacker.png HTTP/1.1', '200', '24461', 'http://10.0.0.21/', 'Mozilla/5.0 (Windows NT 10.0; rv:68.0) Gecko/20100101 Firefox/68.0')
('10.0.0.19', '24/Jun/2020:12:42:29 +0000', 'GET /dirtrav/example1.php?file=hacker.png HTTP/1.1', '200', '24461', 'http://10.0.0.21/', 'Mozilla/5.0 (Windows NT 10.0; rv:68.0) Gecko/20100101 Firefox/68.0')
('10.0.0.19', '24/Jun/2020:12:42:29 +0000', 'GET /dirtrav/example3.php?file=hacker HTTP/1.1', '200', '24462', 'http://10.0.0.21/', 'Mozilla/5.0 (Windows NT 10.0; rv:68.0) Gecko/20100101 Firefox/68.0')
('10.0.0.19', '24/Jun/2020:12:42:29 +0000', 'GET /favicon.ico HTTP/1.1', '200', '14930', '-', 'Mozilla/5.0 (Windows NT 10.0; rv:68.0) Gecko/20100101 Firefox/68.0')
('10.0.0.19', '24/Jun/2020:12:42:32 +0000', 'GET /xml/example1.php?xml=%3Ctest%3Ehacker%3C/test%3E HTTP/1.1', '200', '896', 'http://10.0.0.21/', 'Mozilla/5.0 (Windows NT 10.0; rv:68.0) Gecko/20100101 Firefox/68.0')
('10.0.0.19', '24/Jun/2020:12:43:11 +0000', 'GET /xml/example1.php?xml=%3Ccourse%3Echfl%3C/course%3E HTTP/1.1', '200', '894', '-', 'Mozilla/5.0 (Windows NT 10.0; rv:68.0) Gecko/20100101 Firefox/68.0')
('10.0.0.19', '24/Jun/2020:12:50:07 +0000', 'GET /xml/example1.php?xml=%3C!DOCTYPE%20test%20%5B%3C!ENTITY%20xxe%20SYSTEM%20%22file%3A%2F%2F%2Fetc%2Fpasswd%22%3E%5D%3E%3Ctest%3E%26xxe%3B%3C%2Ftest%3E HTTP/1.1', '200', '1336', '-', 'Mozilla/5.0 (Windows NT 10.0; rv:68.0) Gecko/20100101 Firefox/68.0')
('10.0.0.19', '24/Jun/2020:13:00:28 +0000', 'GET /upload HTTP/1.1', '301', '552', '-', 'Mozilla/5.0 (Windows NT 10.0; rv:68.0) Gecko/20100101 Firefox/68.0')
('10.0.0.19', '24/Jun/2020:13:00:28 +0000', 'GET /upload/ HTTP/1.1', '200', '354', '-', 'Mozilla/5.0 (Windows NT 10.0; rv:68.0) Gecko/20100101 Firefox/68.0')
('10.0.0.19', '24/Jun/2020:13:00:31 +0000', 'GET /uploads/ HTTP/1.1', '404', '497', '-', 'Mozilla/5.0 (Windows NT 10.0; rv:68.0) Gecko/20100101 Firefox/68.0')
('10.0.0.19', '24/Jun/2020:13:01:37 +0000', 'GET /upload/access.log HTTP/1.1', '200', '2233', '-', 'Mozilla/5.0 (Windows NT 10.0; rv:68.0) Gecko/20100101 Firefox/68.0')
('10.0.0.19', '24/Jun/2020:13:10:23 +0000', 'GET /xml/example1.php?xml=%3C!DOCTYPE%20test%20%3C!ENTITY%20xxe%20SYSTEM%20%22file%3A%2F%2F%2Fetc%2Fhosts%22%3E%3E%3Ctest%3E%26xxe%3B%3C%2Ftest%3E HTTP/1.1', '200', '976', '-', 'Mozilla/5.0 (Windows NT 10.0; rv:68.0) Gecko/20100101 Firefox/68.0')
('10.0.0.19', '24/Jun/2020:13:10:42 +0000', 'GET /xml/example1.php? HTTP/1.1', '200', '939', '-', 'Mozilla/5.0 (Windows NT 10.0; rv:68.0) Gecko/20100101 Firefox/68.0')
('10.0.0.19', '24/Jun/2020:13:10:46 +0000', 'GET /xml/ HTTP/1.1', '200', '354', '-', 'Mozilla/5.0 (Windows NT 10.0; rv:68.0) Gecko/20100101 Firefox/68.0')

```

15. Now click on Find icon at the top of the Desktop.





```

root@jason-Virtual-Machine: /home/jason

la/5.0 (Windows NT 10.0; rv:68.0) Gecko/20100101 Firefox/68.0')
('10.0.0.19', '24/Jun/2020:13:10:46 +0000', 'GET /xml/ HTTP/1.1', '200', '354', '-', 'Mozilla/5.0 (Windo
ws NT 10.0; rv:68.0) Gecko/20100101 Firefox/68.0')
('10.0.0.19', '24/Jun/2020:13:10:49 +0000', 'GET / HTTP/1.1', '200', '1860', '-', 'Mozilla/5.0 (Windows
NT 10.0; rv:68.0) Gecko/20100101 Firefox/68.0')
('10.0.0.19', '24/Jun/2020:13:10:49 +0000', 'GET /dirtrav/example1.php?file=hacker.png HTTP/1.1', '200',
'24461', 'http://10.0.0.21/', 'Mozilla/5.0 (Windows NT 10.0; rv:68.0) Gecko/20100101 Firefox/68.0')
('10.0.0.19', '24/Jun/2020:13:10:49 +0000', 'GET /dirtrav/example2.php?file=/var/www/files/hacker.png HT
TP/1.1', '200', '24461', 'http://10.0.0.21/', 'Mozilla/5.0 (Windows NT 10.0; rv:68.0) Gecko/20100101 Fir
efox/68.0')
('10.0.0.19', '24/Jun/2020:13:10:49 +0000', 'GET /dirtrav/example3.php?file=hacker HTTP/1.1', '200', '24
462', 'http://10.0.0.21/', 'Mozilla/5.0 (Windows NT 10.0; rv:68.0) Gecko/20100101 Firefox/68.0')
('10.0.0.19', '24/Jun/2020:13:11:08 +0000', 'GET /sql/example1.php?name=root HTTP/1.1', '200', '950', '
http://10.0.0.21/', 'Mozilla/5.0 (Windows NT 10.0; rv:68.0) Gecko/20100101 Firefox/68.0')
('10.0.0.19', '24/Jun/2020:13:11:17 +0000', 'GET /xml/example2.php?name=hacker HTTP/1.1', '200', '895',
'http://10.0.0.21/', 'Mozilla/5.0 (Windows NT 10.0; rv:68.0) Gecko/20100101 Firefox/68.0')
('10.0.0.19', '24/Jun/2020:13:11:48 +0000', 'GET /upload/access.log HTTP/1.1', '200', '4531', '-', 'Mozil
la/5.0 (Windows NT 10.0; rv:68.0) Gecko/20100101 Firefox/68.0')
('10.0.0.8', '24/Jun/2020:13:24:59 +0000', 'GET / HTTP/1.1', '200', '1861', '-', 'Mozilla/5.0 (X11; Ubun
tu; Linux x86_64; rv:77.0) Gecko/20100101 Firefox/77.0')
('10.0.0.8', '24/Jun/2020:13:25:00 +0000', 'GET /css/bootstrap-responsive.css HTTP/1.1', '200', '4540',
'http://10.0.0.21/', 'Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:77.0) Gecko/20100101 Firefox/77.0')
('10.0.0.8', '24/Jun/2020:13:25:00 +0000', 'GET /css/bootstrap.css HTTP/1.1', '200', '18255', 'http://10
.0.0.21/', 'Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:77.0) Gecko/20100101 Firefox/77.0')
('10.0.0.8', '24/Jun/2020:13:25:00 +0000', 'GET /dirtrav/example2.php?file=/var/www/files/hacker.png HT
TP/1.1', '200', '24461', 'http://10.0.0.21/', 'Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:77.0) Gecko/201
00101 Firefox/77.0')
('10.0.0.8', '24/Jun/2020:13:25:00 +0000', 'GET /dirtrav/example1.php?file=hacker.png HTTP/1.1', '200',
'24461', 'http://10.0.0.21/', 'Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:77.0) Gecko/20100101 Firefox/7
7.0')
('10.0.0.8', '24/Jun/2020:13:25:00 +0000', 'GET /dirtrav/example3.php?file=hacker HTTP/1.1', '200', '244
62', 'http://10.0.0.21/', 'Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:77.0) Gecko/20100101 Firefox/77.0')
('10.0.0.8', '24/Jun/2020:13:25:00 +0000', 'GET /favicon.ico HTTP/1.1', '200', '14930', '-', 'Mozilla/5.
0 (X11; Ubuntu; Linux x86_64; rv:77.0) Gecko/20100101 Firefox/77.0')
('10.0.0.8', '24/Jun/2020:13:25:03 +0000', 'GET /commandexec/example1.php?ip=127.0.0.1 HTTP/1.1', '200',
'1060', 'http://10.0.0.21/', 'Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:77.0) Gecko/20100101 Firefox/7
7.0')
('10.0.0.8', '24/Jun/2020:13:25:03 +0000', 'GET /commandexec/example1.php?ip=127.0.0.1 HTTP/1.1', '200',
'1060', 'http://10.0.0.21/', 'Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:77.0) Gecko/20100101 Firefox/7
7.0')

```

16. Now, we will apply an encoded variant of the root' Union Select filter to look for the signs of an SQL Injection attack. To apply the encoded filter, type root %27%20%20union%20select in the Find box and press Enter.

18. Upon examining the top log entry, we find that it contains a GET request with a UniON SeLeCT query. Furthermore, the UniON SeLeCT query is referencing parameters such as 1,table\_name,3,4,5 From Information\_schema.tables where Table\_Schema=Database() limit 0,1-- - etc. From this, we can infer that an attacker has tried to gain access to the details of tables present in the database and the contents stored within them by using the UniON SeLeCT query, which is an indicator of an SQL Injection attack.

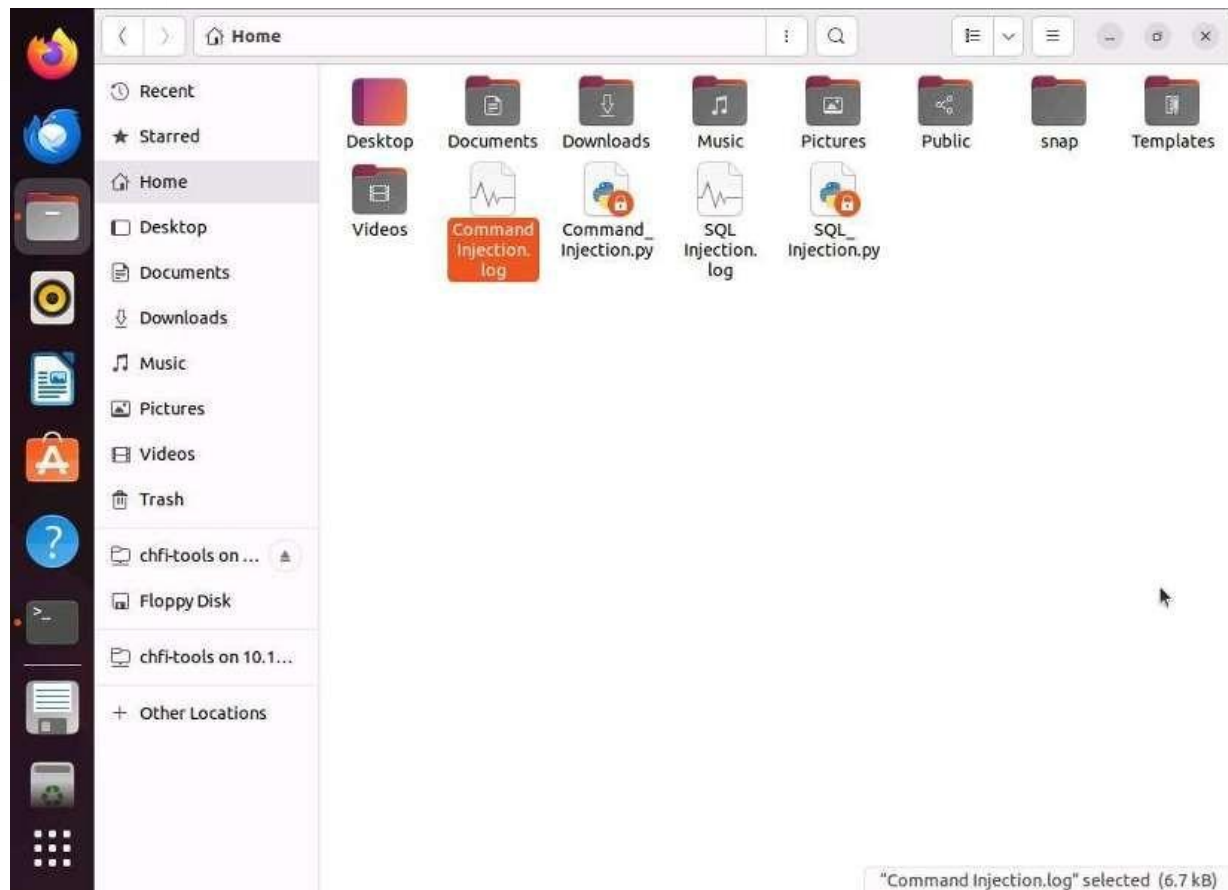


```

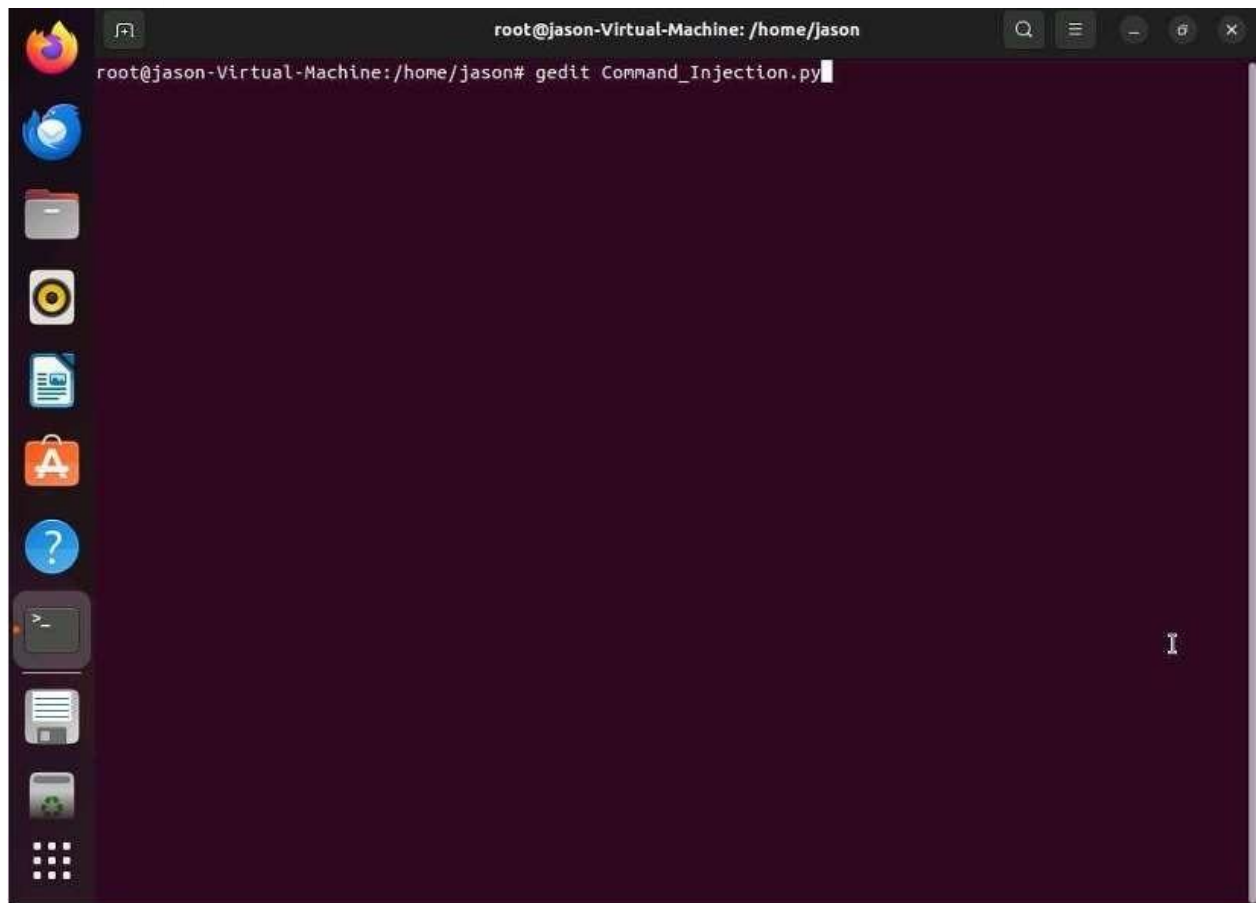
root@jason-Virtual-Machine: /home/jason
- 'Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:77.0) Gecko/20100101 Firefox/77.0')
('10.0.0.19', '24/Jun/2020:14:46:09 +0000', 'GET /sql/example1.php?name=root HTTP/1.1', '200', '950', '
http://10.0.0.21/', 'Mozilla/5.0 (Windows NT 10.0; rv:68.0) Gecko/20100101 Firefox/68.0')
('10.0.0.19', '24/Jun/2020:15:22:28 +0000', 'GET /sql/example1.php?name=root%27%20or%20%271%27=%271 HTTP
P/1.1', '200', '985', '-', 'Mozilla/5.0 (Windows NT 10.0; rv:68.0) Gecko/20100101 Firefox/68.0')
('10.0.0.19', '24/Jun/2020:15:38:34 +0000', 'GET /sql/example1.php?name=root%27%20--%20- HTTP/1.1', '20
0', '950', '-', 'Mozilla/5.0 (Windows NT 10.0; rv:68.0) Gecko/20100101 Firefox/68.0')
('10.0.0.19', '24/Jun/2020:15:39:26 +0000', 'GET /sql/example1.php?name=root%27%20%20Union%20SeLeCT%201
,2,3,4,5%20%20--%20- HTTP/1.1', '200', '961', '-', 'Mozilla/5.0 (Windows NT 10.0; rv:68.0) Gecko/2010010
1 Firefox/68.0')
('10.0.0.19', '24/Jun/2020:15:52:36 +0000', 'GET /sql/example1.php?name=root%27%20%20Union%20SeLeCT%201
, database(),3,4,5%20%20--%20- HTTP/1.1', '200', '966', '-', 'Mozilla/5.0 (Windows NT 10.0; rv:68.0) Geck
o/20100101 Firefox/68.0')
('10.0.0.19', '24/Jun/2020:15:52:49 +0000', 'GET /sql/example1.php?name=root%27%20%20Union%20SeLeCT%201
, tables(),3,4,5%20%20--%20- HTTP/1.1', '200', '885', '-', 'Mozilla/5.0 (Windows NT 10.0; rv:68.0) Gecko/
20100101 Firefox/68.0')
('10.0.0.19', '24/Jun/2020:15:53:14 +0000', 'GET /sql/example1.php?name=root%27%20%20Union%20SeLeCT%201
, version(),3,4,5%20%20--%20- HTTP/1.1', '200', '974', '-', 'Mozilla/5.0 (Windows NT 10.0; rv:68.0) Gecko
/20100101 Firefox/68.0')
('10.0.0.19', '24/Jun/2020:15:58:37 +0000', 'GET /sql/example1.php?name=root%27%20%20Union%20SeLeCT%201
, table_name,3,4,5%20From%20Information_schema.tables%20where%20Table_Schema=DatabaseT()%20--%20- HTTP/1.1
', '200', '886', '-', 'Mozilla/5.0 (Windows NT 10.0; rv:68.0) Gecko/20100101 Firefox/68.0')
('10.0.0.19', '24/Jun/2020:15:59:01 +0000', 'GET /sql/example1.php?name=root%27%20%20Union%20SeLeCT%201
, table_name,3,4,5%20From%20Information_schema.tables%20where%20Table_Schema=DatabaseT()%20--%20- HTTP/1.1
', '200', '964', '-', 'Mozilla/5.0 (Windows NT 10.0; rv:68.0) Gecko/20100101 Firefox/68.0')
('10.0.0.19', '24/Jun/2020:15:59:40 +0000', 'GET /sql/example1.php?name=root%27%20%20Union%20SeLeCT%201
, table_name,3,4,5%20From%20Information_schema.tables%20where%20Table_Schema=DatabaseT()%20limit%201,2--%2
0- HTTP/1.1', '200', '950', '-', 'Mozilla/5.0 (Windows NT 10.0; rv:68.0) Gecko/20100101 Firefox/68.0')
('10.0.0.19', '24/Jun/2020:15:59:56 +0000', 'GET /sql/example1.php?name=root%27%20%20Union%20SeLeCT%201
, table_name,3,4,5%20From%20Information_schema.tables%20where%20Table_Schema=DatabaseT()%20limit%201,1--%2
0- HTTP/1.1', '200', '950', '-', 'Mozilla/5.0 (Windows NT 10.0; rv:68.0) Gecko/20100101 Firefox/68.0')
('10.0.0.19', '24/Jun/2020:16:00:02 +0000', 'GET /sql/example1.php?name=root%27%20%20Union%20SeLeCT%201
, table_name,3,4,5%20From%20Information_schema.tables%20where%20Table_Schema=DatabaseT()%20limit%203,1--%2
0- HTTP/1.1', '200', '930', '-', 'Mozilla/5.0 (Windows NT 10.0; rv:68.0) Gecko/20100101 Firefox/68.0')
('10.0.0.19', '24/Jun/2020:16:00:25 +0000', 'GET /sql/example1.php?name=root%27%20%20Union%20SeLeCT%201
, table_name,3,4,5%20From%20Information_schema.tables%20where%20Table_Schema=DatabaseT()%20limit%200,1--%2
0- HTTP/1.1', '200', '950', '-', 'Mozilla/5.0 (Windows NT 10.0; rv:68.0) Gecko/20100101 Firefox/68.0')
root@jason-Virtual-Machine: /home/jason#

```

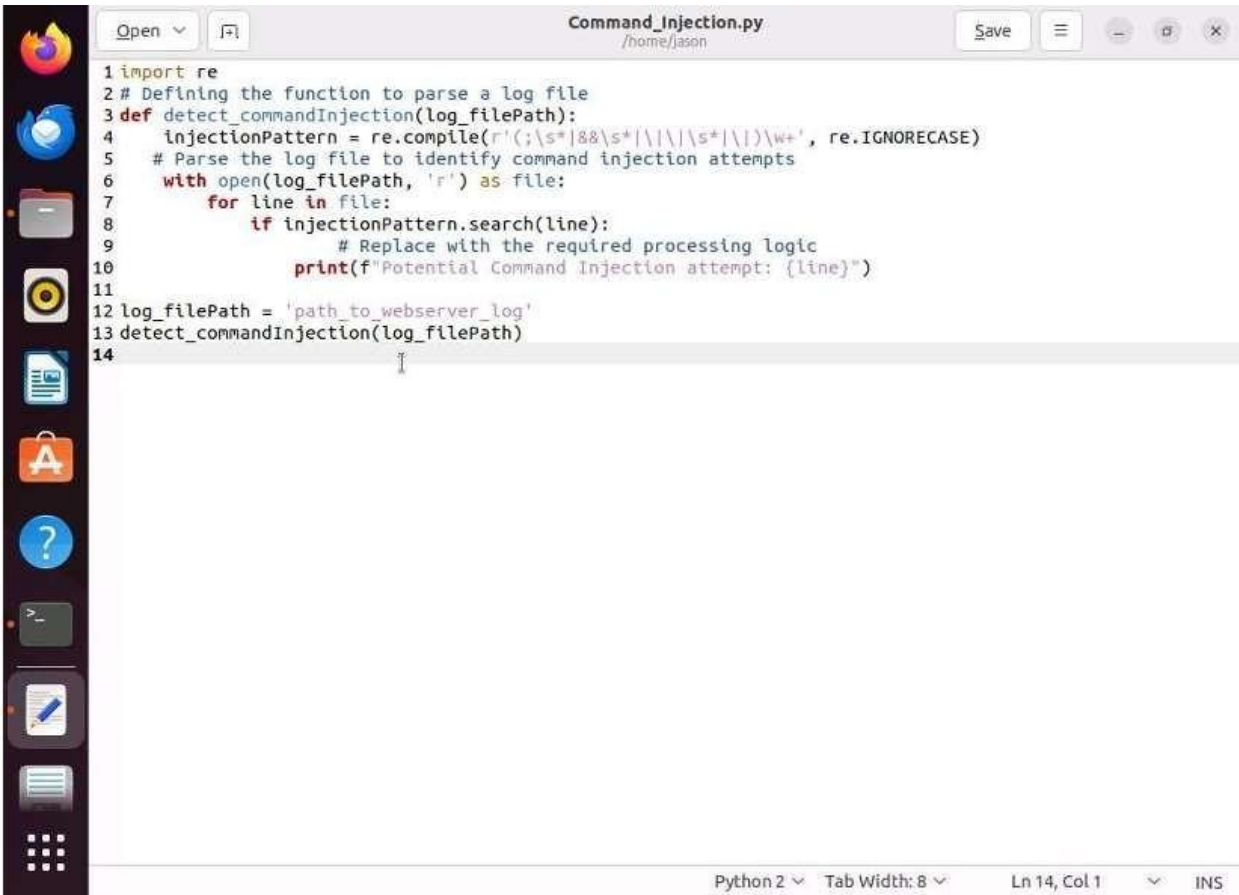
19. Before continuing with the next task, follow the Steps from 6 and 7 to navigate to Evidence Files\Log Files\Apache Logs directory and copy the Command Injection.log file and paste it in the Home directory.



20. Now, we will examine Web Server Logs for a command injection attack using Python. To do so, in the Terminal window, type `gedit Command_Injection.py` and press Enter.



21. Command\_Injection.py text editor will be opened; Now type the code shown in the screenshot below into the file and save it in the Home directory.

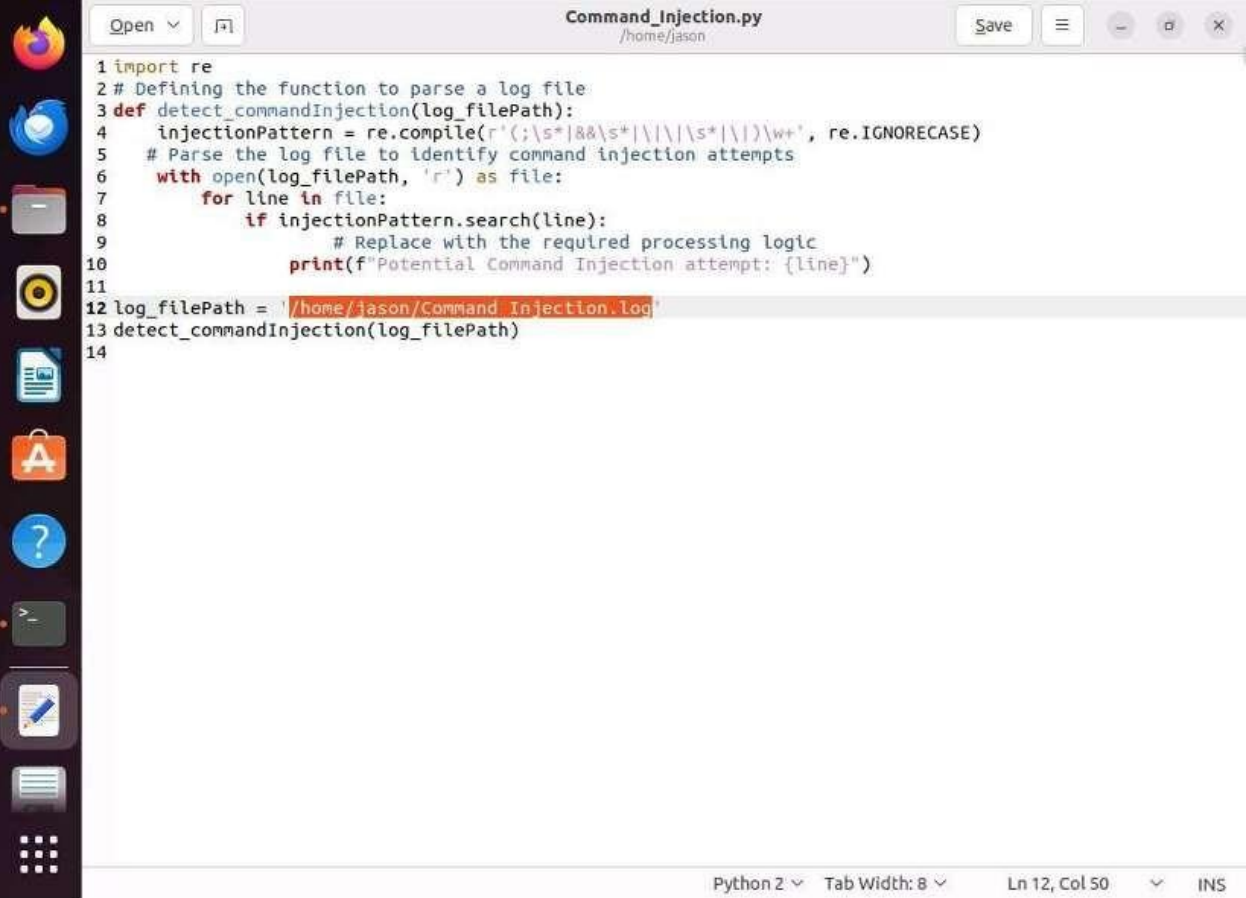


```
1 import re
2 # Defining the function to parse a log file
3 def detect_commandInjection(log_filePath):
4     injectionPattern = re.compile(r';\s*|&&\s*|\\\s*|\\\s*|\\\s*|\\\s*', re.IGNORECASE)
5     # Parse the log file to identify command injection attempts
6     with open(log_filePath, 'r') as file:
7         for line in file:
8             if injectionPattern.search(line):
9                 # Replace with the required processing logic
10                print(f"Potential Command Injection attempt: {line}")
11
12 log_filePath = 'path_to_webserver_log'
13 detect_commandInjection(log_filePath)
14
```

Python 2 ▾ Tab Width: 8 ▾ Ln 14, Col 1 ▾ INS

22. We need to provide the log file location in line 12 of the Python script to parse the logs.

23. In line 15 replace `path_to_webserver_log` with `/home/jason/Command Injection.log` as shown in the screenshot. Save it and close the Text Editor window.



The screenshot shows a Linux desktop with a dark theme. On the left is a vertical dock with icons for Firefox, Telegram, a file manager, a media player, a help icon, a terminal, a text editor, and a grid icon. The main window is a code editor titled 'Command\_Injection.py' with the path '/home/jason' shown below the title. The editor contains Python code for detecting command injection. The status bar at the bottom indicates 'Python 2', 'Tab Width: 8', 'Ln 12, Col 50', and 'INS'.

```
1 import re
2 # Defining the function to parse a log file
3 def detect_commandInjection(log_filePath):
4     injectionPattern = re.compile(r'(;|&&|s*|\\\\|\\s*|\\|)w+', re.IGNORECASE)
5     # Parse the log file to identify command injection attempts
6     with open(log_filePath, 'r') as file:
7         for line in file:
8             if injectionPattern.search(line):
9                 # Replace with the required processing logic
10                print(f"Potential Command Injection attempt: {line}")
11
12 log_filePath = '/home/jason/Command Injection.log'
13 detect_commandInjection(log_filePath)
14
```

24. Now, switch back to the terminal window, and type `python3 Command_Injection.py` and press Enter to run the Python script.

25. Upon successful execution of the script, you can view the parsed Command Injection logs in the terminal.



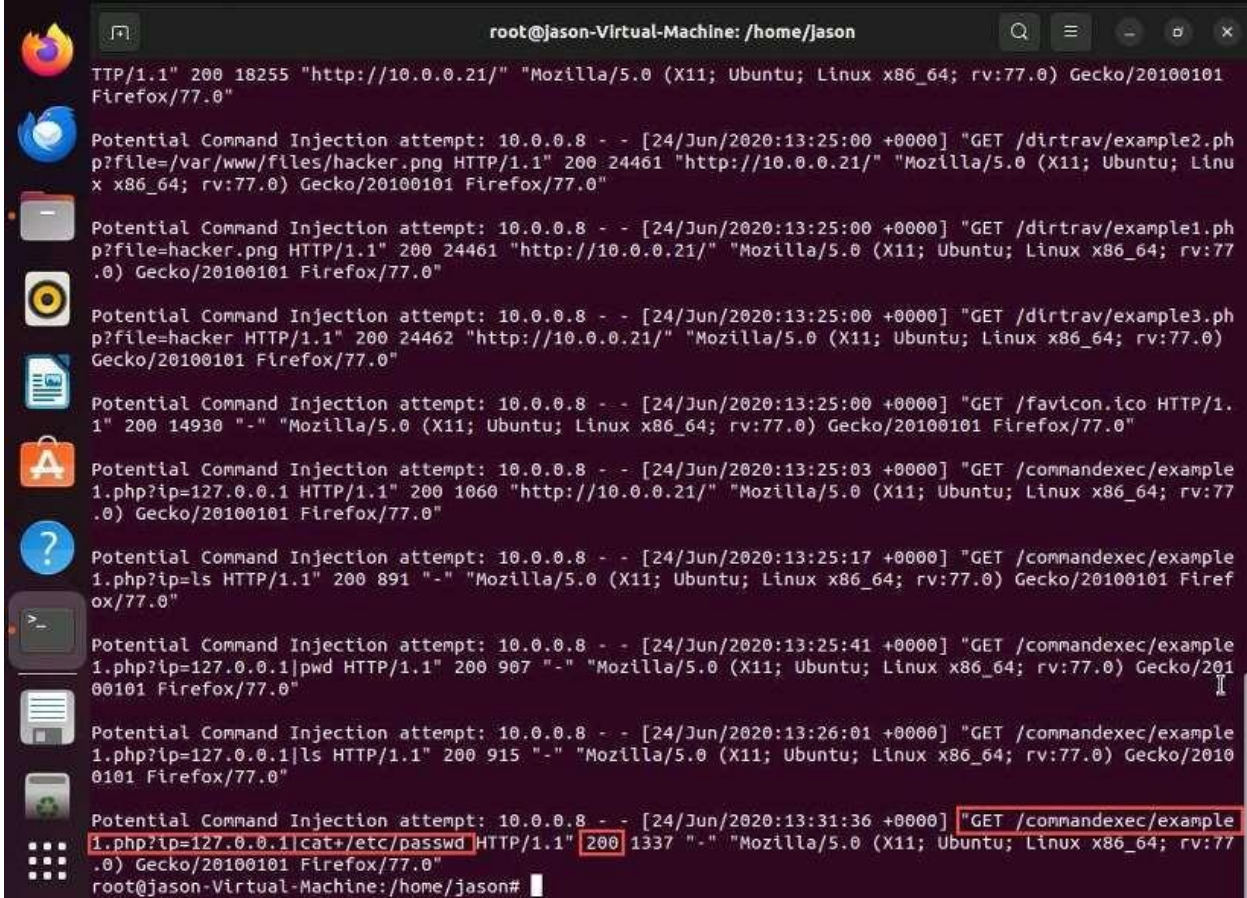
```

root@jason-Virtual-Machine: /home/jason# python3 Command_Injection.py
Potential Command Injection attempt: 10.0.0.19 - - [24/Jun/2020:12:42:29 +0000] "GET / HTTP/1.1" 200 186
1 "-" Mozilla/5.0 (Windows NT 10.0; rv:68.0) Gecko/20100101 Firefox/68.0"
Potential Command Injection attempt: 10.0.0.19 - - [24/Jun/2020:12:42:29 +0000] "GET /css/bootstrap-resp
onsive.css HTTP/1.1" 200 4540 "http://10.0.0.21/" Mozilla/5.0 (Windows NT 10.0; rv:68.0) Gecko/20100101
Firefox/68.0"
Potential Command Injection attempt: 10.0.0.19 - - [24/Jun/2020:12:42:29 +0000] "GET /css/bootstrap.css
HTTP/1.1" 200 18255 "http://10.0.0.21/" Mozilla/5.0 (Windows NT 10.0; rv:68.0) Gecko/20100101 Firefox/6
8.0"
Potential Command Injection attempt: 10.0.0.19 - - [24/Jun/2020:12:42:29 +0000] "GET /dirtrav/example2.p
hp?file=/var/www/files/hacker.png HTTP/1.1" 200 24461 "http://10.0.0.21/" Mozilla/5.0 (Windows NT 10.0;
rv:68.0) Gecko/20100101 Firefox/68.0"
Potential Command Injection attempt: 10.0.0.19 - - [24/Jun/2020:12:42:29 +0000] "GET /dirtrav/example1.p
hp?file=hacker.png HTTP/1.1" 200 24461 "http://10.0.0.21/" Mozilla/5.0 (Windows NT 10.0; rv:68.0) Gecko
/20100101 Firefox/68.0"
Potential Command Injection attempt: 10.0.0.19 - - [24/Jun/2020:12:42:29 +0000] "GET /dirtrav/example3.p
hp?file=hacker HTTP/1.1" 200 24462 "http://10.0.0.21/" Mozilla/5.0 (Windows NT 10.0; rv:68.0) Gecko/201
00101 Firefox/68.0"
Potential Command Injection attempt: 10.0.0.19 - - [24/Jun/2020:12:42:29 +0000] "GET /favicon.ico HTTP/1
.1" 200 14930 "-" Mozilla/5.0 (Windows NT 10.0; rv:68.0) Gecko/20100101 Firefox/68.0"
Potential Command Injection attempt: 10.0.0.19 - - [24/Jun/2020:12:42:32 +0000] "GET /xml/example1.php?x
ml=%3Ctest%3Ehacker%3C/test%3E HTTP/1.1" 200 896 "http://10.0.0.21/" Mozilla/5.0 (Windows NT 10.0; rv:6
8.0) Gecko/20100101 Firefox/68.0"
Potential Command Injection attempt: 10.0.0.19 - - [24/Jun/2020:12:43:11 +0000] "GET /xml/example1.php?x
ml=%3Ccourse%3Echfi%3C/course%3E HTTP/1.1" 200 894 "-" Mozilla/5.0 (Windows NT 10.0; rv:68.0) Gecko/201
00101 Firefox/68.0"
Potential Command Injection attempt: 10.0.0.19 - - [24/Jun/2020:12:50:07 +0000] "GET /xml/example1.php?x
ml=%3C!DOCTYPE%20test%20%5B%3C!ENTITY%20xxe%20SYSTEM%20%22file%3A%2F%2F%2Fetc%2Fpasswd%22%3E%5D%3E%3Ctes
t%3E%26xxe%3B%3C%2Ftest%3E HTTP/1.1" 200 1336 "-" Mozilla/5.0 (Windows NT 10.0; rv:68.0) Gecko/20100101

```

26. Scroll to the bottom of the log entries and you will see malicious query used in the attack: GET /commandexec/example1.php?ip=127.0.0.1|cat+/etc/passwd with HTTP status code 200: This denotes that the request made to the server has been processed.





```

root@jason-Virtual-Machine: /home/jason

TTP/1.1" 200 18255 "http://10.0.0.21/" "Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:77.0) Gecko/20100101 Firefox/77.0"

Potential Command Injection attempt: 10.0.0.8 - - [24/Jun/2020:13:25:00 +0000] "GET /dirtrav/example2.php?file=/var/www/files/hacker.png HTTP/1.1" 200 24461 "http://10.0.0.21/" "Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:77.0) Gecko/20100101 Firefox/77.0"

Potential Command Injection attempt: 10.0.0.8 - - [24/Jun/2020:13:25:00 +0000] "GET /dirtrav/example1.php?file=hacker.png HTTP/1.1" 200 24461 "http://10.0.0.21/" "Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:77.0) Gecko/20100101 Firefox/77.0"

Potential Command Injection attempt: 10.0.0.8 - - [24/Jun/2020:13:25:00 +0000] "GET /dirtrav/example3.php?file=hacker HTTP/1.1" 200 24462 "http://10.0.0.21/" "Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:77.0) Gecko/20100101 Firefox/77.0"

Potential Command Injection attempt: 10.0.0.8 - - [24/Jun/2020:13:25:00 +0000] "GET /favicon.ico HTTP/1.1" 200 14930 "-" "Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:77.0) Gecko/20100101 Firefox/77.0"

Potential Command Injection attempt: 10.0.0.8 - - [24/Jun/2020:13:25:03 +0000] "GET /commandexec/example1.php?ip=127.0.0.1 HTTP/1.1" 200 1060 "http://10.0.0.21/" "Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:77.0) Gecko/20100101 Firefox/77.0"

Potential Command Injection attempt: 10.0.0.8 - - [24/Jun/2020:13:25:17 +0000] "GET /commandexec/example1.php?ip=ls HTTP/1.1" 200 891 "-" "Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:77.0) Gecko/20100101 Firefox/77.0"

Potential Command Injection attempt: 10.0.0.8 - - [24/Jun/2020:13:25:41 +0000] "GET /commandexec/example1.php?ip=127.0.0.1|pwd HTTP/1.1" 200 907 "-" "Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:77.0) Gecko/20100101 Firefox/77.0"

Potential Command Injection attempt: 10.0.0.8 - - [24/Jun/2020:13:26:01 +0000] "GET /commandexec/example1.php?ip=127.0.0.1|ls HTTP/1.1" 200 915 "-" "Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:77.0) Gecko/20100101 Firefox/77.0"

Potential Command Injection attempt: 10.0.0.8 - - [24/Jun/2020:13:31:36 +0000] "GET /commandexec/example1.php?ip=127.0.0.1|cat+/etc/passwd HTTP/1.1" 200 1337 "-" "Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:77.0) Gecko/20100101 Firefox/77.0"

root@jason-Virtual-Machine: /home/jason#

```

27. In this manner, you can examine the log files from Apache for indicators pertaining to various web attacks using Python. 28. Close all open Windows.